Tema 04 — RISC-V

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 10: RISC-V

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), José Mendoza (jfmendoza@uc.cl)

Pregunta 1: Elabore el código I

Elabore, utilizando el Assembly RISC-V, un programa que a partir de un arreglo `arr` de largo `len`, determine si este posee un par de números **en posiciones distintas** tal que su suma sea igual a un valor `pair_sum`. Si se encuentra un par de números que cumple lo pedido, se deben guardar los índices del par en las variables `x_pair_sum` e `y_pair_sum`; en otro caso, sus valores deben ser iguales a -1. Si existe más de un par de valores que cumple lo pedido, puede escoger arbitrariamente los primeros que encuentre.

A continuación, tres ejemplos:

$$\texttt{arr} = [1, 3, 5, 7, -5] \quad \texttt{pair_sum} = 10 \rightarrow (\texttt{x_pair_sum}, \texttt{y_pair_sum}) = \quad (1, 3)$$
$$\texttt{arr} = [1, 3, 5, 7, -5] \quad \texttt{pair_sum} = 11 \rightarrow (\texttt{x_pair_sum}, \texttt{y_pair_sum}) = \quad (-1, -1)$$
$$\texttt{arr} = [1, 3, 5, 7, -5] \quad \texttt{pair_sum} = 14 \rightarrow (\texttt{x_pair_sum}, \texttt{y_pair_sum}) = \quad (-1, -1)$$

Puede utilizar el siguiente fragmento de código como base:

```
.data
  arr:          .word 1, 3, 5, 7, -5
  len:          .word 5
  pair_sum:     .word 10
  x_pair_sum:   .word -1
  y_pair_sum:   .word -1
.text
  # Su código aquí.
```

Pregunta 2: Explique el código

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_x_n`:

```
.data
  x:            .word 7
  n:            .word 3
  result:       .word 0
.text
    lw a0, x
    lw a1, n
    la s0, result
    addi s1, zero, 1
    addi sp, sp, -4
    sw ra, 0(sp)
    jal ra, func_x_n
    lw ra, 0(sp)
    addi sp, sp, 4
    sw a0, 0(s0)
    li a7, 10
    ecall
  func_x_n:
    beq a1, s1, func_x_n_end
    addi sp, sp, -8
    sw ra, 0(sp)
    sw a0, 4(sp)
    addi a1, a1, -1
    jal ra, func_x_n
    lw ra, 0(sp)
    lw t0, 4(sp)
    addi sp, sp, 8
    mul a0, a0, t0
    func_x_n_end:
      jalr zero, 0(ra)
```

Este fragmento representa el cómputo de una función $f(x, n)$. A partir de este:

1. Indique, con argumentos y en términos de x y n, lo que retorna la función $f(x, n)$. Por ejemplo, $f(x, n) = x + n$.
2. Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.

Pregunta 3: Elabore el código II

Estamos en el año 2084. Eres el ingeniero principal de la Resistencia y has recibido un cargamento de suministros médicos vitales. El sistema de transporte automático ha dejado las cajas en el muelle de carga, pero están completamente desordenadas según su número de prioridad.

Para que los drones de distribución puedan recoger las cajas de forma eficiente, el protocolo exige que los suministros estén ordenados de **menor a mayor prioridad** en la memoria central del sistema. Debido a que el procesador del almacén es un modelo antiguo basado en la arquitectura **RISC-V** y que por la antigüedad se ha estropeado una parte del procesador, dando error al ejecutar loops, deberas programar una rutina totalmente recursiva.

El Algoritmo: Bubble Sort con Doble Recursión

En esta versión avanzada para el procesador del almacén, utilizaremos una **doble recursión** para garantizar que el sistema opere de manera puramente funcional. El problema se divide en dos procesos recursivos anidados:

1. **Recursión Externa (Control de Pasadas):** Esta función se encarga de determinar cuántas veces debemos revisar el arreglo.
   - **Caso Base:** Si el tamaño del fragmento a ordenar es $n \leq 1$, el proceso termina.
   - **Llamada Recursiva:** En cada paso, invoca a la función de la pasada interna y luego se llama a sí misma con un tamaño de $n - 1$.
2. **Recursión Interna (Pasada de Comparación):** Sustituye al bucle de recorrido tradicional. Se encarga de comparar el elemento en el índice i con el de la posición $i + 1$.
   - **Caso Base:** Si el índice i alcanza el límite superior $(n - 1)$, la pasada termina y el control regresa a la función externa.
   - **Acción:** Si $arr[i] > arr[i + 1]$, se realiza el intercambio en memoria.
   - **Llamada Recursiva:** La función se llama a sí misma incrementando el índice $(i + 1)$ para comparar la siguiente pareja.

Esta arquitectura permite que el procesador gestione el ordenamiento exclusivamente mediante la manipulación del **Stack Pointer** (`sp`) y el registro de retorno (`ra`).

Código Base

Utiliza la siguiente estructura de memoria para comenzar tu implementación:

```
.data
arr:    .word 64, 34, 25, 12, 22, 11, 90  # Prioridades de las cajas
size:   .word 7                           # Cantidad de cajas
.text
main:
  # Su código aquí
```

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 10: RISC-V

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), José Mendoza (jfmendoza@uc.cl)

Pregunta 1: Elabore el código I

Elabore, utilizando el Assembly RISC-V, un programa que a partir de un arreglo `arr` de largo `len`, determine si este posee un par de números **en posiciones distintas** tal que su suma sea igual a un valor `pair_sum`. Si se encuentra un par de números que cumple lo pedido, se deben guardar los índices del par en las variables `x_pair_sum` e `y_pair_sum`; en otro caso, sus valores deben ser iguales a -1. Si existe más de un par de valores que cumple lo pedido, puede escoger arbitrariamente los primeros que encuentre.

A continuación, tres ejemplos:

$$\texttt{arr} = [1, 3, 5, 7, -5] \quad \texttt{pair_sum} = 10 \rightarrow (\texttt{x_pair_sum}, \texttt{y_pair_sum}) = \quad (1, 3)$$
$$\texttt{arr} = [1, 3, 5, 7, -5] \quad \texttt{pair_sum} = 11 \rightarrow (\texttt{x_pair_sum}, \texttt{y_pair_sum}) = \quad (-1, -1)$$
$$\texttt{arr} = [1, 3, 5, 7, -5] \quad \texttt{pair_sum} = 14 \rightarrow (\texttt{x_pair_sum}, \texttt{y_pair_sum}) = \quad (-1, -1)$$

Puede utilizar el siguiente fragmento de código como base:

```
.data
  arr:          .word 1, 3, 5, 7, -5
  len:          .word 5
  pair_sum:     .word 10
  x_pair_sum:   .word -1
  y_pair_sum:   .word -1
.text
  # Su código aquí.
```

Solución: A continuación, un código que cumple lo pedido:

```
.data
  arr:         .word 1, 3, 5, 7, -5
  len:         .word 5
  pair_sum:    .word 10
  x_pair_sum: .word -1
  y_pair_sum: .word -1
.text
  la s0, arr                          # Dirección inicial del arreglo.
  addi s1, zero, 4                    # Constante para multiplicar índices por 4 para obtener direcciones.
  lw s2, len                          # Largo del arreglo.
  addi s3, s2, -1                     # len - 1 para terminar antes el loop de i.
  lw s4, pair_sum                     # Suma a buscar.
  la s5, x_pair_sum                   # Dirección para almacenar el índice del primer número del par.
  la s6, y_pair_sum                   # Dirección para almacenar el índice del segundo número del par.
  addi t0, zero, 0                    # Índice i
  find_pair_sum:
    i_loop:                           # Iteración respecto al índice i
      addi t1, t0, 1                  # j = i + 1, no vemos los elementos hacia atrás, ya se revisaron.
      j_loop:
        mul t2, t0, s1                # t2 = 4 * i
        mul t3, t1, s1                # t3 = 4 * j
        add t2, s0, t2                # t2 = dir(arr) + 4 * i
        add t3, s0, t3                # t3 = dir(arr) + 4 * j
        lw t2, 0(t2)                  # t2 = arr[i]
        lw t3, 0(t3)                  # t3 = arr[j]
        add t4, t2, t3                # t4 = arr[i] + arr[j]
        beq t4, s4, pair_sum_found    # arr[i] + arr[j] == pair_sum
      j_loop_continue:
        addi t1, t1, 1
        blt t1, s2, j_loop            # if j >= n, end j_loop
    i_loop_continue:
      addi t0, t0, 1
      bge t0, s3, end                 # if i >= n - 1, end i_loop
      jal zero, i_loop
    pair_sum_found:
      sw t0, 0(s5)                    # x_pair_sum = i
      sw t1, 0(s6)                    # y_pair_sum = j
  end:
    addi a7, zero, 10
    ecall
```

Pregunta 2: Explique el código

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_x_n`:

```
.data
  x:            .word 7
  n:            .word 3
  result:       .word 0
.text
    lw a0, x
    lw a1, n
    la s0, result
    addi s1, zero, 1
    addi sp, sp, -4
    sw ra, 0(sp)
    jal ra, func_x_n
    lw ra, 0(sp)
    addi sp, sp, 4
    sw a0, 0(s0)
    li a7, 10
    ecall
  func_x_n:
    beq a1, s1, func_x_n_end
    addi sp, sp, -8
    sw ra, 0(sp)
    sw a0, 4(sp)
    addi a1, a1, -1
    jal ra, func_x_n
    lw ra, 0(sp)
    lw t0, 4(sp)
    addi sp, sp, 8
    mul a0, a0, t0
    func_x_n_end:
      jalr zero, 0(ra)
```

Este fragmento representa el cómputo de una función $f(x, n)$. A partir de este:

1. Indique, con argumentos y en términos de x y n, lo que retorna la función $f(x, n)$. Por ejemplo, $f(x, n) = x + n$.

   Solución: La función computa la potencia de x a la n, *i.e.* $f(x, n) = x^n$. Esto se evidencia en el hecho de que el valor de retorno, almacenado en el registro `a0`, es igual a $x \times f(x, n-1)$, con caso base $f(x, 1) = x$. De esta forma, se tiene que:

   $$f(x, n) = x \times f(x, n-1) = x \times x \times f(x, n-2) = \cdots = x \times x \times x \times \cdots \times x = x^n$$

2. Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.

   Solución: El fragmento anterior **sí respeta** la convención de llamadas de RISC-V. Los motivos son los siguientes:

   - Se respalda `ra` antes de cada llamado.

- Se usan los registros `a0` y `a1` para los argumentos de `func_x_n`, y `a0` para su retorno.
- Se respalda el registro `a0` antes de cada llamado recursivo, considerando su sobre-escritura; su uso posterior al llamado; y el hecho de ser *caller-saved*.

Pregunta 3: Elabore el código II

Estamos en el año 2084. Eres el ingeniero principal de la Resistencia y has recibido un cargamento de suministros médicos vitales. El sistema de transporte automático ha dejado las cajas en el muelle de carga, pero están completamente desordenadas según su número de prioridad.

Para que los drones de distribución puedan recoger las cajas de forma eficiente, el protocolo exige que los suministros estén ordenados de **menor a mayor prioridad** en la memoria central del sistema. Debido a que el procesador del almacén es un modelo antiguo basado en la arquitectura **RISC-V** y que por la antigüedad se ha estropeado una parte del procesador, dando error al ejecutar loops, deberas programar una rutina totalmente recursiva.

El Algoritmo: Bubble Sort con Doble Recursión

En esta versión avanzada para el procesador del almacén, utilizaremos una **doble recursión** para garantizar que el sistema opere de manera puramente funcional. El problema se divide en dos procesos recursivos anidados:

1. **Recursión Externa (Control de Pasadas):** Esta función se encarga de determinar cuántas veces debemos revisar el arreglo.
   - **Caso Base:** Si el tamaño del fragmento a ordenar es $n \leq 1$, el proceso termina.
   - **Llamada Recursiva:** En cada paso, invoca a la función de la pasada interna y luego se llama a sí misma con un tamaño de $n - 1$.
2. **Recursión Interna (Pasada de Comparación):** Sustituye al bucle de recorrido tradicional. Se encarga de comparar el elemento en el índice i con el de la posición $i + 1$.
   - **Caso Base:** Si el índice i alcanza el límite superior $(n - 1)$, la pasada termina y el control regresa a la función externa.
   - **Acción:** Si $arr[i] > arr[i + 1]$, se realiza el intercambio en memoria.
   - **Llamada Recursiva:** La función se llama a sí misma incrementando el índice $(i + 1)$ para comparar la siguiente pareja.

Esta arquitectura permite que el procesador gestione el ordenamiento exclusivamente mediante la manipulación del **Stack Pointer** (`sp`) y el registro de retorno (`ra`).

Código Base

Utiliza la siguiente estructura de memoria para comenzar tu implementación:

```
.data
arr:    .word 64, 34, 25, 12, 22, 11, 90  # Prioridades de las cajas
size:   .word 7                           # Cantidad de cajas
.text
main:
  # Su código aquí
```

Solución: A continuación, un código que cumple lo pedido:

```
.data
arr:    .word 64, 34, 25, 12, 22, 11, 90  # Arreglo desordenado
size:   .word 7                           # Tamano del arreglo
.text
main:
    la   a0, arr               # a0 = Direccion base del arreglo
    lw   a1, size              # a1 = n (elementos totales)

    jal  ra, bubble_sort       # Llamada a la recursion externa

    addi a7, zero, 10
    ecall

bubble_sort:
    addi s0, zero, 1           # s0 = 1 (para comparacion)
    ble  a1, s0, sort_end      # Si n <= 1, terminar

    addi sp, sp, -4            # Reservar espacio en Stack
    sw   ra, 0(sp)             # Guardar ra

    addi a2, zero, 0           # a2 = indice i = 0
    addi a1, a1, -1            # a3 = limite = n - 1
    jal  ra, bubble_pass       # Llamada a la recursion interna

    lw   ra, 0(sp)             # Restaurar ra
    addi sp, sp, 4             # Liberar Stack

    addi sp, sp, -4            # Reservar espacio en Stack
    sw   ra, 0(sp)             # Guardar ra
    jal  ra, bubble_sort       # Siguiente pasada recursiva
    lw   ra, 0(sp)             # Restaurar ra
    addi sp, sp, 4             # Liberar Stack

sort_end:
    jalr zero, 0(ra)

bubble_pass:
    bge  a2, a1, pass_end      # Si i >= limite, terminar pasada

    slli t0, a2, 2             # t0 = i * 4 (offset)
    add  t0, t0, a0            # t0 = &arr[i]
    lw   t1, 0(t0)             # t1 = arr[i]
    lw   t2, 4(t0)             # t2 = arr[i+1]

    ble  t1, t2, no_swap       # Si arr[i] <= arr[i+1], no swap
    sw   t2, 0(t0)             # Intercambiar: arr[i] = t2
    sw   t1, 4(t0)             # Intercambiar: arr[i+1] = t1
no_swap:
    addi sp, sp, -4            # Guardar ra para bajar nivel
    sw   ra, 0(sp)             # de recursion interna

    addi a2, a2, 1             # i = i + 1
    jal  ra, bubble_pass       # Llamada recursiva interna

    lw   ra, 0(sp)             # Restaurar ra
    addi sp, sp, 4             # Liberar Stack
pass_end:
    jalr zero, 0(ra)
```

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 9: RISC-V

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), Fernanda Escobar (fer_jez@uc.cl)

Pregunta 1: Encuesta de Casas de Hogwarts

Usted desea determinar cuál de las cuatro casas de Hogwarts es la mejor basándose en una encuesta a 12 amigos. Cada amigo votó por una opción (1: Gryffindor, 2: Ravenclaw, 3: Hufflepuff, 4: Slytherin).

Implemente un programa en RISC-V que recorra la lista de votos almacenada en memoria, cuente las frecuencias de cada casa y finalmente imprima el número de la casa ganadora.

Resultados: $[4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4]$

Paso Intermedio (Python): Antes de programar en ensamblador, revise la lógica en Python para ayudarte a pensar

```python
# Lógica intermedia para la encuesta
votos = [4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4]
# Contadores para casas 1, 2, 3 y 4
conteos = [0, 0, 0, 0]

# Contar frecuencias
for v in votos:
    conteos[v-1] += 1

# Encontrar el ganador (ID de la casa)
max_votos = 0
ganador = 0
for i in range(4):
    if conteos[i] > max_votos:
        max_votos = conteos[i]
        ganador = i + 1

print(ganador)
```

Hint: Utilice la directiva `.word` para almacenar los votos en el segmento `.data`. Dado que cada instrucción y palabra ocupa 4 bytes, recuerde ajustar sus desplazamientos de memoria en consecuencia.

Pregunta 2: Explique el código

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_n_m`:

```
.data
n:   .word 37
m:   .word 5
q:   .word 0

.text
main:
  addi sp, sp, -16
  sw   ra, 12(sp)

  la   t0, n
  lw   a0, 0(t0)
  la   t1, m
  lw   a1, 0(t1)

  jal  ra, func_n_m

  la   t2, q
  sw   a0, 0(t2)

  lw   ra, 12(sp)
  addi sp, sp, 16
  addi a7, zero, 10
  ecall

func_n_m:
  blt  a0, a1, ccc

  addi sp, sp, -16
  sw   ra, 12(sp)

  sub  a0, a0, a1
  jal  ra, func_n_m
  addi a0, a0, 1

  lw   ra, 12(sp)
  addi sp, sp, 16
  jalr x0, 0(ra)

ccc:
  addi a0, x0, 0
  jalr x0, 0(ra)
```

Este fragmento representa el cómputo de una función $f(n, m)$. A partir de este:

1. (1.5 ptos.) Indique, con argumentos y en términos de n y m, lo que retorna la función $f(n, m)$. Por ejemplo, $f(n, m) = n + m$.
2. (1.5 ptos.) Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.

Pregunta 3: Elabore el código

Elabore, utilizando el Assembly RISC-V, un programa que, dado un arreglo `arr` de largo `len` cuyos elementos son enteros de 32 bits (`.word`), determine si este es un "palíndromo" (1) o no (0), es decir, que si se invierte el orden de sus elementos, el arreglo no cambia. El resultado se debe computar en la variable `is_palindrome`.

A continuación, tres ejemplos:

$$\begin{aligned} \texttt{arr} = [1,3,3,1] &\rightarrow \texttt{is_palindrome} = 1 \\ \texttt{arr} = [1,2,3,2,1] &\rightarrow \texttt{is_palindrome} = 1 \\ \texttt{arr} = [1,2,3,4,5,6] &\rightarrow \texttt{is_palindrome} = 0 \end{aligned}$$

Puede utilizar el siguiente fragmento de código como base:

```
.data
    arr:            .word 1,3,3,1  # Arreglo
    len:            .word 4        # Largo del arreglo
    is_palindrome:  .word -1       # Es palíndromo => TRUE = 1, FALSE = 0
.text
  # Su código aquí
```

IMPORTANTE: No es necesario que respete la convención RISC-V en este ejercicio.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Ayudantía 9: RISC-V

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), Fernanda Escobar (fer_jez@uc.cl)

Pregunta 1: Encuesta de Casas de Hogwarts

Usted desea determinar cuál de las cuatro casas de Hogwarts es la mejor basándose en una encuesta a 12 amigos. Cada amigo votó por una opción (1: Gryffindor, 2: Ravenclaw, 3: Hufflepuff, 4: Slytherin).

Implemente un programa en RISC-V que recorra la lista de votos almacenada en memoria, cuente las frecuencias de cada casa y finalmente imprima el número de la casa ganadora.

Resultados: $[4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4]$

Paso Intermedio (Python): Antes de programar en ensamblador, revise la lógica en Python para ayudarte a pensar

```python
# Lógica intermedia para la encuesta
votos = [4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4]
# Contadores para casas 1, 2, 3 y 4
conteos = [0, 0, 0, 0]

# Contar frecuencias
for v in votos:
    conteos[v-1] += 1

# Encontrar el ganador (ID de la casa)
max_votos = 0
ganador = 0
for i in range(4):
    if conteos[i] > max_votos:
        max_votos = conteos[i]
        ganador = i + 1

print(ganador)
```

Hint: Utilice la directiva `.word` para almacenar los votos en el segmento `.data`. Dado que cada instrucción y palabra ocupa 4 bytes, recuerde ajustar sus desplazamientos de memoria en consecuencia.

Solución:

```
.data
votos:      .word 4, 3, 1, 4, 2, 2, 1, 3, 4, 1, 2, 4
conteos:    .word 0, 0, 0, 0  # Array de contadores

.text
main:
    la t0, votos        # t0: puntero a votos (temporary)
    la t1, conteos      # t1: puntero a conteos (temporary)
    addi t2, zero, 0            # t2: i = 0
    addi t3, zero, 12           # t3: N = 12 amigos

loop_votos:
    beq t2, t3, init_max # Si i == 12, terminar conteo
    lw t4, 0(t0)         # t4 = votos[i]

    # Calcular dirección: conteos + (votos[i]-1)*4
    addi t4, t4, -1     # Ajustar casa a índice 0-3
    slli t4, t4, 2      # t4 = t4 * 4 bytes
    add t5, t1, t4      # t5 = dirección final del contador

    lw t6, 0(t5)        # t6 = conteo actual
    addi t6, t6, 1      # t6++
    sw t6, 0(t5)        # Guardar conteo

    addi t0, t0, 4      # Siguiente palabra en votos
    addi t2, t2, 1      # i++
    jal zero, loop_votos

init_max:
    addi t2, zero, 0            # Reiniciar i para buscar ganador
    addi s0, zero, -1           # s0: max_votos (saved register)
    addi s1, zero, 0            # s1: casa_ganadora (saved register)
    la t1, conteos      # Reiniciar puntero a conteos

loop_max:
    addi t3, zero, 4
    beq t2, t3, print_res
    lw t4, 0(t1)        # t4 = conteos[i]

    # Si conteos[i] <= max_votos, saltar
    bge s0, t4, next_iter
    addi s0, t4, 0          # s0 = nuevo max_votos
    addi s1, t2, 1      # s1 = casa ganadora (i + 1)

next_iter:
    addi t1, t1, 4      # Siguiente contador
    addi t2, t2, 1      # i++
    jal zero, loop_max

print_res:
    addi a0, s1, 0          # a0: Argumento para imprimir (ID casa)
    addi a7, zero, 1            # a7: Código syscall PrintInt
    ecall

    addi a7, zero, 10           # a7: Código syscall Exit
    ecall
```

Pregunta 2: Explique el código

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_n_m`:

```
.data
n:   .word 37
m:   .word 5
q:   .word 0

.text
main:
  addi sp, sp, -16
  sw   ra, 12(sp)

  la   t0, n
  lw   a0, 0(t0)
  la   t1, m
  lw   a1, 0(t1)

  jal  ra, func_n_m

  la   t2, q
  sw   a0, 0(t2)

  lw   ra, 12(sp)
  addi sp, sp, 16
  addi a7, zero, 10
  ecall

func_n_m:
  blt  a0, a1, ccc

  addi sp, sp, -16
  sw   ra, 12(sp)

  sub  a0, a0, a1
  jal  ra, func_n_m
  addi a0, a0, 1

  lw   ra, 12(sp)
  addi sp, sp, 16
  jalr x0, 0(ra)

ccc:
  addi a0, x0, 0
  jalr x0, 0(ra)
```

Este fragmento representa el cómputo de una función $f(n, m)$. A partir de este:

1. (1.5 ptos.) Indique, con argumentos y en términos de n y m, lo que retorna la función $f(n, m)$. Por ejemplo, $f(n, m) = n + m$.

   Solución: La función implementa la **división entera por restas sucesivas**, por lo que:

   $$f(n, m) = n//m$$

   En el código, el registro `a0` contiene el argumento n y el registro `a1` contiene el argumento m. La subrutina `func_n_m` resta repetidamente m a n hasta que el valor de $a0$ es menor

que $a1$, alcanzando el caso base. En cada retorno desde la recursión se incrementa en una unidad el valor acumulado en `a0`, representando la cantidad de veces que m cabe en n. De esta forma, el resultado final (el cociente entero de la división) queda almacenado en `a0`, y posteriormente se guarda en la etiqueta `q`.

2. (1.5 ptos.) Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.

Solución: El fragmento anterior **NO respeta** la convención de llamadas de RISC-V.

Aunque el programa ejecuta correctamente la división entera y maneja el `stack` sin errores de ejecución, desde el punto de vista de la convención formal incumple reglas esenciales. En particular:

- El registro `a1` es de tipo *caller-saved*. La función `func_n_m` realiza un llamado recursivo sin respaldar previamente este registro, lo que constituye una violación directa a la convención. Al ser una función recursiva, actúa simultáneamente como *caller* y *callee*, por lo que debe preservar el valor de sus argumentos antes del llamado.
- Los registros `a0` y `a1` no se respaldan en el programa principal (`main`) antes de realizar el llamado a la subrutina, a pesar de que ambos pueden ser modificados por esta. Esto también infringe la responsabilidad del *caller*.
- Los registros `t0`, `t1` y `t2` son de tipo *caller-saved* y se utilizan en el programa principal sin respaldo previo. Aunque en este caso no generan un error funcional, formalmente representan una falta a la convención.
- Además, el código emplea los nombres `x0` en lugar de los nombres simbólicos `zero`. Según la convención de llamadas de RISC-V, se exige el uso de nombres simbólicos estandarizados (`zero`, `ra`, `sp`, `a0-a7`, etc.) en lugar de los nombres numéricos (`x0`, `x1`, ...). Este uso constituye una infracción formal de la convención, aunque no afecte la ejecución.

Pregunta 3: Elabore el código

Elabore, utilizando el Assembly RISC-V, un programa que, dado un arreglo `arr` de largo `len` cuyos elementos son enteros de 32 bits (`.word`), determine si este es un "palíndromo" (1) o no (0), es decir, que si se invierte el orden de sus elementos, el arreglo no cambia. El resultado se debe computar en la variable `is_palindrome`.

A continuación, tres ejemplos:

$$\begin{aligned} \texttt{arr} = [1,3,3,1] &\rightarrow \texttt{is_palindrome} = 1 \\ \texttt{arr} = [1,2,3,2,1] &\rightarrow \texttt{is_palindrome} = 1 \\ \texttt{arr} = [1,2,3,4,5,6] &\rightarrow \texttt{is_palindrome} = 0 \end{aligned}$$

Puede utilizar el siguiente fragmento de código como base:

```
.data
    arr:            .word 1,3,3,1  # Arreglo
    len:            .word 4        # Largo del arreglo
    is_palindrome:  .word -1       # Es palíndromo => TRUE = 1, FALSE = 0
.text
  # Su código aquí
```

IMPORTANTE: No es necesario que respete la convención RISC-V en este ejercicio.

Solución: En la siguiente plana, se muestra una solución que utiliza dos índices `i`, `j` tales que inicialmente `i = 0` y `j = len - 1`. Luego, se realiza el siguiente procedimiento iterativo:

- Si `i >= j`, el programa termina y el arreglo se considera palíndromo.
- Si no ocurre lo anterior, se revisa si `arr[i] == arr[j]`. Si no se cumple la condición, el programa termina y el arreglo no se considera palíndromo.
- Si la condición se cumple, se repite la iteración con nuevos índices: `i += 1`, `j -= 1`.

```
.data
    arr:             .word 1,2,5,3,2,1  # Arreglo
    len:             .word 6            # Largo del arreglo
    is_palindrome:   .word -1           # Es palíndromo => TRUE = 1, FALSE = 0
.text
  la s0, arr               # Dirección de memoria del arreglo.
  la s1, is_palindrome     # Dirección de memoria del resultado.
  addi s2, zero, 4         # Constante para computar dirección de memoria.
  addi s3, zero, 1         # Constante para cargar un 1 en is_palindrome.
  addi t0, zero, 0         # Índice i.
  lw t1, len
  addi t1, t1, -1          # Índice j.
  loop:
    bge t0, t1, is_true    # Si i >= j, recorrimos todo el arreglo y el arreglo es palíndromo.
    mul a0, t0, s2         # a0 = 4*i
    add a0, a0, s0         # a0 = dirección de memoria arr[i]
    lw a0, 0(a0)           # a0 = arr[i]
    mul a1, t1, s2         # a1 = 4*j
    add a1, a1, s0         # a1 = dirección de memoria arr[j]
    lw a1, 0(a1)           # a1 = arr[j]
    bne a0, a1, is_false   # Si a[i] != a[j], el arreglo no es palíndromo
    addi t0, t0, 1         # i += 1
    addi t1, t1, -1        # j -= 1
    jal zero, loop
  is_true:
    sw s3, 0(s1)           # is_palindrome = 1
    jal zero, end
  is_false:                # is_palindrome = 0
    sw zero, 0(s1)
  end:
    addi a7, zero, 10
    ecall
```

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2025)

Guía de ejercicios: RISC-V

Ayudantes: Daniela Ríos (danielaarp@uc.cl), Alberto Maturana (alberto.maturana@uc.cl), Ignacio Gajardo (gajardo.ignacio@uc.cl)

Pregunta 1: DCCálculo - RISC-V al rescate

Te encuentras estudiando para un ramo de la Universidad, cuando en un ejercicio de ayudantía te aparece la temible función **factorial:** `x!`, la cual te toma mucho tiempo calcular y no alcanzarás a terminar tu estudio a tiempo.

Por suerte, también te encuentras cursando Arquitectura de Computadores y, como expert@ en RISC-V, decides elaborar un código capaz de calcular esta función al instante y así agilizar tu estudio. Para esto, puedes utilizar el siguiente código base:

```
.data
  x: .word 4

.text
  # Completa tu código acá,
```

IMPORTANTE: No es necesario que respetes la convención de llamadas en este ejercicio.

Solución: A continuación se presenta una alternativa de código que implementa la función f(x) = x!:

```
.data
        x: .word 8

.text
    lw t0, x              # Contador de cuántas multiplicaciones nos faltan
    addi s0, zero, 1      # Constante = 1
    addi sp, sp, -4       # Respaldar ra
    sw ra, 0(sp)
    jal ra, factorial_x   # Llamada a subrutina
    lw ra, 0(sp)

    # ecall para imprimir el resultado (código 1)
    addi a7, zero, 1      # a7 -> Código de la operación a realizar
    add a0, zero, t0      # a0 -> (Si es necesario) Argumentos de la operación a realizar
    ecall

    # ecall para terminar el programa (código 10)
    addi a7, zero, 10
    ecall

    factorial_x:
        beq t0, s0, end_factorial_x
        addi sp, sp, -8
        sw ra, 0(sp)          # Respaldo de ra
        sw t0, 4(sp)          # Guardar qué números tengo que multiplicar
        addi t0,t0, -1        # Decrezco el contador
        jal ra, factorial_x   # Vuelvo a llamar a la subrutina
        lw ra, 0(sp)          # Recuperamos ra

        # Multiplicación
        lw t1, 4(sp)           # Recuperamos el número que toca multiplicar ahora
        addi sp, sp, 8         # Reestablecemos el SP
        mul t0, t0, t1         # t0 = t0 * t1

        end_factorial_x:
            jalr zero, 0(ra)  # Equivalente a RET, sale de la función
```

Puede ver el desarrollo de este código en la siguiente cápsula.

Nota: Si bien el código funciona correctamente, este **NO** cumple con la convención de llamadas de RISC-V, dado que se debería haber hecho uso del registro `a0` para almacenar el argumento x y el valor de retorno de la subrutina `factorial_x`. Además, dentro de la subrutina se utilizó el registro `t0` (*caller-saved*), sin hacer el respaldo correspondiente.

Pregunta 2: DCCálculo - Resumen perdido

Llegó el día de dar la prueba para la que tanto has estudiado y, al mirar Canvas, ¡te enteras que podrás llevar una hoja de resumen! Sin embargo, entre todo el caos del estudio no logras encontrar la hoja en tu cajón de apuntes. Por suerte, tienes todas tus hojas marcadas con un identificador, por lo que decides utilizar tus habilidades de RISC-V y con ellas crear un programa que te ayude a encontrar el resumen.

Para esto, deberás elaborar un programa que realice la **búsqueda binaria** de un elemento `x` sobre un arreglo ordenado `arr` y guardar su índice en `element_index`. Si el elemento `x` no se encuentra en `arr`, entonces se debe mantener por defecto el valor `element_index` = -1.

La búsqueda binaria se realiza a través del siguiente procedimiento:

- Se revisa el elemento central del arreglo `arr[left_bound : right_bound]`. Llamaremos `c` al índice del elemento central y se computa así: `c = (left_bound + right_bound) // 2`.
- Si el valor del elemento central es igual al valor buscado, *i.e.* `arr[c] == x`, se retorna su posición como el restulado de la búsqueda.
- Si su valor no es igual al buscado:
  - Si el elemento central es menor al elemento buscado, se realiza la búsqueda nuevamente con `left_bound` igual a `c + 1`.
  - Si el elemento central es mayor al elemento buscado, se realiza la búsqueda nuevamente con `right_bound` igual a `c - 1`.
- La búsqueda termina cuando `left_bound` es mayor a `right_bound`.

Puedes utilizar el siguiente fragmento de código como base:

```
.data
    arr:            .word -1000, -255, -7, -1, 0, 10, 11, 27, 255, 1000, 10000  # Arreglo
    left_bound:     .word 0                                                     # Límite izquierdo
    right_bound:    .word 10                                                    # Límite derecho
    x:              .word -7                                                    # Elemento a buscar
    element_index:  .word -1                                                    # Índice del elemento

.text
    # Tu código aquí
```

IMPORTANTE: No es necesario que respetes la convención de llamadas en este ejercicio.

Solución: A continuación se presenta un código que implementa la búsqueda binaria:

```
.data
    arr:            .word -1000, -255, -7, -1, 0, 10, 11, 27, 255, 1000, 10000  # Arreglo
    left_bound:     .word 0                                                     # Límite izquierdo
    right_bound:    .word 10                                                    # Límite derecho
    x:              .word -7                                                    # Elemento a buscar
    element_index:  .word -1                                                    # Índice del elemento

.text
    main:
        # Almacenar elementos en registros
```

```
    # Datos en memoria (.data)
    la a0, arr  # Dirección de memoria del primer elemento del arreglo
    lw a1, left_bound
    lw a2, right_bound
    lw a3, x

    # Constantes útiles
    addi s0, zero, 2  # s0 = 2
    addi s1, zero, 4  # s1 = 4


    # Llamado inicial a la subrutina
    # respaldo de registros
    addi sp, sp, -20  # hacemos espacio en el stack
    sw ra, 0(sp)
    sw a0, 4(sp)
    sw a1, 8(sp)
    sw a2, 12(sp)
    sw a3, 16(sp)

    jal ra, bin_search
    la t0, element_index  # t0 = dirección de memoria de element_index
    sw a0, 0(t0)  # guardamos el índice encontrado (a0) en element_index

    # Terminar el programa -> ecalls (terminar el programa)
    # a7 -> codigo de la operación a realizar
    # a0 -> argumentos de la operación/retornos de la operación
    addi a7, zero, 10  # código 10 = terminar el programa
    ecall

bin_search:
    # 0) Condición de salida: Si left_bound > right_bound, no encontré el elemento
    blt a2, a1, element_not_found

    # 1) Revisar el elemento central
    # Calcular el índice c del elemento central
    add t0, a1, a2  # t0 = left_bound + right_bound
    div t0, t0, s0  # t0 = t0 // 2 = c
    mv t2, t0       # t2 = t0 = c

    # Revisar el elemento asociado al índice c
    # En este punto [t0 = c (índice del arreglo)] y [a0 = dirección de memoria de arr[0]]
    # Recordar que en RISC-V, cada bloque de memoria usa 4 bytes de memoria
    # En otras palabras, si el elemento 0 del array (arr[0]) está en la dirección 0=4*0, entonces
    # el elemento 1 (arr[1]) está en la dirección 4=4*1, el elemento 2 está en la dirección 8=4*2...

    mul t0, t0, s1  # t0= t0*4 (dir. de memoria que tengo que avanzar desde arr[0] hasta arr[c])
    add t0, t0, a0  # t0 = dirección de memoria del elemento arr[c]
    lw t0, 0(t0)    # t0 = arr[c] = 10

    # 2) Si arr[c] == x, encontre el elemento y retorno el índice
    beq t0, a3, element_found

    bgt t0, a3, left_array_search
    # 3) Si arr[c] < x, busco hacia la derecha
    right_array_search:
        add a1, zero, t2  # left_bound = c
        addi a1, a1, 1    # left_bound = c + 1
        beq zero, zero, next_search

    # 4) Si arr[c] > x, busco hacia la izquierda
    left_array_search:
        add a2, zero, t2  # right_bound = c
        addi a2, a2, -1   # right_bound = c - 1

    next_search:  # llamado recursivo a la subrutina bin_search
```

```
            # respaldar antes del siguiente llamado
            addi sp, sp, -28  # generar el espacio en el stack
            sw ra, 0(sp)
            sw a0, 4(sp)
            sw a1, 8(sp)
            sw a2, 12(sp)
            sw a3, 16(sp)
            sw t0, 20(sp)
            sw t2, 24(sp)

            jal ra, bin_search  # llamado recursivo

            # recuperar después del llamado recursivo
            lw ra, 0(sp)
            # a0 NO se recupera, porque el llamado recursivo encontró el resultado
            lw a1, 8(sp)
            lw a2, 12(sp)
            lw a3, 16(sp)
            lw t0, 20(sp)
            lw t2, 24(sp)
            addi sp, sp, 28  # liberamos el espacio en el stack

            beq zero, zero, bin_search_end

        element_found:
            add a0, zero, t2  # a0 = c (el indice del valor que buscabamos)
            beq zero, zero, bin_search_end

        element_not_found:
            addi a0, zero, -1  # a0 = -1 (no se encontró el elemento x)

        bin_search_end:
            jalr zero, 0(ra) # RET, termina la subrutina
```

Puede ver el desarrollo de este código en la siguiente cápsula.

Nota: Si bien el código funciona correctamente, este **NO** cumple con la convención de llamadas de RISC-V, dado que no se hacen los respaldos *caller-saved* y *callee-saved* que propone la convención.

1. *Feedback* cápsulas

Escanee el QR para entregar *feedback* sobre las cápsulas.



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2025)

Guía de ejercicios: RISC-V

Ayudantes: Daniela Ríos (danielaarp@uc.cl), Alberto Maturana (alberto.maturana@uc.cl), Ignacio Gajardo (gajardo.ignacio@uc.cl)

Pregunta 1: DCCálculo - RISC-V al rescate

Te encuentras estudiando para un ramo de la Universidad, cuando en un ejercicio de ayudantía te aparece la temible función **factorial:** `x!`, la cual te toma mucho tiempo calcular y no alcanzarás a terminar tu estudio a tiempo.

Por suerte, también te encuentras cursando Arquitectura de Computadores y, como expert@ en RISC-V, decides elaborar un código capaz de calcular esta función al instante y así agilizar tu estudio. Para esto, puedes utilizar el siguiente código base:

```
.data
  x: .word 4

.text
  # Completa tu código acá,
```

IMPORTANTE: No es necesario que respetes la convención de llamadas en este ejercicio.

Solución: A continuación se presenta una alternativa de código que implementa la función f(x) = x!:

```
.data
        x: .word 8

.text
    lw t0, x              # Contador de cuántas multiplicaciones nos faltan
    addi s0, zero, 1      # Constante = 1
    addi sp, sp, -4       # Respaldar ra
    sw ra, 0(sp)
    jal ra, factorial_x   # Llamada a subrutina
    lw ra, 0(sp)

    # ecall para imprimir el resultado (código 1)
    addi a7, zero, 1      # a7 -> Código de la operación a realizar
    add a0, zero, t0      # a0 -> (Si es necesario) Argumentos de la operación a realizar
    ecall

    # ecall para terminar el programa (código 10)
    addi a7, zero, 10
    ecall

    factorial_x:
        beq t0, s0, end_factorial_x
        addi sp, sp, -8
        sw ra, 0(sp)          # Respaldo de ra
        sw t0, 4(sp)          # Guardar qué números tengo que multiplicar
        addi t0,t0, -1        # Decrezco el contador
        jal ra, factorial_x   # Vuelvo a llamar a la subrutina
        lw ra, 0(sp)          # Recuperamos ra

        # Multiplicación
        lw t1, 4(sp)           # Recuperamos el número que toca multiplicar ahora
        addi sp, sp, 8         # Reestablecemos el SP
        mul t0, t0, t1         # t0 = t0 * t1

        end_factorial_x:
            jalr zero, 0(ra)  # Equivalente a RET, sale de la función
```

Puede ver el desarrollo de este código en la siguiente cápsula.

Nota: Si bien el código funciona correctamente, este **NO** cumple con la convención de llamadas de RISC-V, dado que se debería haber hecho uso del registro `a0` para almacenar el argumento x y el valor de retorno de la subrutina `factorial_x`. Además, dentro de la subrutina se utilizó el registro `t0` (*caller-saved*), sin hacer el respaldo correspondiente.

Pregunta 2: DCCálculo - Resumen perdido

Llegó el día de dar la prueba para la que tanto has estudiado y, al mirar Canvas, ¡te enteras que podrás llevar una hoja de resumen! Sin embargo, entre todo el caos del estudio no logras encontrar la hoja en tu cajón de apuntes. Por suerte, tienes todas tus hojas marcadas con un identificador, por lo que decides utilizar tus habilidades de RISC-V y con ellas crear un programa que te ayude a encontrar el resumen.

Para esto, deberás elaborar un programa que realice la **búsqueda binaria** de un elemento `x` sobre un arreglo ordenado `arr` y guardar su índice en `element_index`. Si el elemento `x` no se encuentra en `arr`, entonces se debe mantener por defecto el valor `element_index` = -1.

La búsqueda binaria se realiza a través del siguiente procedimiento:

- Se revisa el elemento central del arreglo `arr[left_bound : right_bound]`. Llamaremos `c` al índice del elemento central y se computa así: `c = (left_bound + right_bound) // 2`.
- Si el valor del elemento central es igual al valor buscado, *i.e.* `arr[c] == x`, se retorna su posición como el restulado de la búsqueda.
- Si su valor no es igual al buscado:
  - Si el elemento central es menor al elemento buscado, se realiza la búsqueda nuevamente con `left_bound` igual a `c + 1`.
  - Si el elemento central es mayor al elemento buscado, se realiza la búsqueda nuevamente con `right_bound` igual a `c - 1`.
- La búsqueda termina cuando `left_bound` es mayor a `right_bound`.

Puedes utilizar el siguiente fragmento de código como base:

```
.data
    arr:            .word -1000, -255, -7, -1, 0, 10, 11, 27, 255, 1000, 10000  # Arreglo
    left_bound:     .word 0                                                     # Límite izquierdo
    right_bound:    .word 10                                                    # Límite derecho
    x:              .word -7                                                    # Elemento a buscar
    element_index:  .word -1                                                    # Índice del elemento

.text
    # Tu código aquí
```

IMPORTANTE: No es necesario que respetes la convención de llamadas en este ejercicio.

Solución: A continuación se presenta un código que implementa la búsqueda binaria:

```
.data
    arr:            .word -1000, -255, -7, -1, 0, 10, 11, 27, 255, 1000, 10000  # Arreglo
    left_bound:     .word 0                                                     # Límite izquierdo
    right_bound:    .word 10                                                    # Límite derecho
    x:              .word -7                                                    # Elemento a buscar
    element_index:  .word -1                                                    # Índice del elemento

.text
    main:
        # Almacenar elementos en registros
```

```
    # Datos en memoria (.data)
    la a0, arr  # Dirección de memoria del primer elemento del arreglo
    lw a1, left_bound
    lw a2, right_bound
    lw a3, x

    # Constantes útiles
    addi s0, zero, 2  # s0 = 2
    addi s1, zero, 4  # s1 = 4


    # Llamado inicial a la subrutina
    # respaldo de registros
    addi sp, sp, -20  # hacemos espacio en el stack
    sw ra, 0(sp)
    sw a0, 4(sp)
    sw a1, 8(sp)
    sw a2, 12(sp)
    sw a3, 16(sp)

    jal ra, bin_search
    la t0, element_index  # t0 = dirección de memoria de element_index
    sw a0, 0(t0)  # guardamos el índice encontrado (a0) en element_index

    # Terminar el programa -> ecalls (terminar el programa)
    # a7 -> codigo de la operación a realizar
    # a0 -> argumentos de la operación/retornos de la operación
    addi a7, zero, 10  # código 10 = terminar el programa
    ecall

bin_search:
    # 0) Condición de salida: Si left_bound > right_bound, no encontré el elemento
    blt a2, a1, element_not_found

    # 1) Revisar el elemento central
    # Calcular el índice c del elemento central
    add t0, a1, a2  # t0 = left_bound + right_bound
    div t0, t0, s0  # t0 = t0 // 2 = c
    mv t2, t0       # t2 = t0 = c

    # Revisar el elemento asociado al índice c
    # En este punto [t0 = c (índice del arreglo)] y [a0 = dirección de memoria de arr[0]]
    # Recordar que en RISC-V, cada bloque de memoria usa 4 bytes de memoria
    # En otras palabras, si el elemento 0 del array (arr[0]) está en la dirección 0=4*0, entonces
    # el elemento 1 (arr[1]) está en la dirección 4=4*1, el elemento 2 está en la dirección 8=4*2...

    mul t0, t0, s1  # t0= t0*4 (dir. de memoria que tengo que avanzar desde arr[0] hasta arr[c])
    add t0, t0, a0  # t0 = dirección de memoria del elemento arr[c]
    lw t0, 0(t0)    # t0 = arr[c] = 10

    # 2) Si arr[c] == x, encontre el elemento y retorno el índice
    beq t0, a3, element_found

    bgt t0, a3, left_array_search
    # 3) Si arr[c] < x, busco hacia la derecha
    right_array_search:
        add a1, zero, t2  # left_bound = c
        addi a1, a1, 1    # left_bound = c + 1
        beq zero, zero, next_search

    # 4) Si arr[c] > x, busco hacia la izquierda
    left_array_search:
        add a2, zero, t2  # right_bound = c
        addi a2, a2, -1   # right_bound = c - 1

    next_search:  # llamado recursivo a la subrutina bin_search
```

```
            # respaldar antes del siguiente llamado
            addi sp, sp, -28  # generar el espacio en el stack
            sw ra, 0(sp)
            sw a0, 4(sp)
            sw a1, 8(sp)
            sw a2, 12(sp)
            sw a3, 16(sp)
            sw t0, 20(sp)
            sw t2, 24(sp)

            jal ra, bin_search  # llamado recursivo

            # recuperar después del llamado recursivo
            lw ra, 0(sp)
            # a0 NO se recupera, porque el llamado recursivo encontró el resultado
            lw a1, 8(sp)
            lw a2, 12(sp)
            lw a3, 16(sp)
            lw t0, 20(sp)
            lw t2, 24(sp)
            addi sp, sp, 28  # liberamos el espacio en el stack

            beq zero, zero, bin_search_end

        element_found:
            add a0, zero, t2  # a0 = c (el indice del valor que buscabamos)
            beq zero, zero, bin_search_end

        element_not_found:
            addi a0, zero, -1  # a0 = -1 (no se encontró el elemento x)

        bin_search_end:
            jalr zero, 0(ra) # RET, termina la subrutina
```

Puede ver el desarrollo de este código en la siguiente cápsula.

Nota: Si bien el código funciona correctamente, este **NO** cumple con la convención de llamadas de RISC-V, dado que no se hacen los respaldos *caller-saved* y *callee-saved* que propone la convención.

Pregunta 3: FizzBuzz

Para esta pregunta deberá programar una pequeña versión del juego FizzBuzz en RISC-V. Para esto, recibirá en la sección *.data* una etiqueta n, que corresponde a un entero positivo, y deberá recorrer los números desde 1 hasta n.

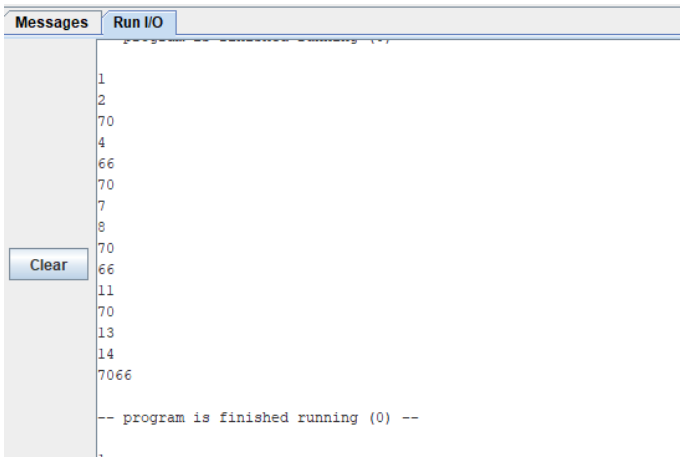
Para cada número:

- Si el número es divisible por 3 y 5, imprima 7066.
- Si el número es divisible solo por 3, imprima 70.
- Si el número es divisible solo por 5, imprima 66.
- En cualquier otro caso, imprima el número original.

Cada resultado debe imprimirse en una línea distinta. Para esto, cuenta con una subrutina *newline* ya implementada, la cual imprime un salto de línea.

```
newline:
    addi a0, zero, 10
    addi a7, zero, 11
    ecall
```

A modo de ejemplo, si $n = 15$, el resultado a imprimir será:



IMPORTANTE: No es necesario respetar la convención de llamada en este ejercicio.

Puede utilizar el siguiente fragmento de código como base:

```
.data
    n:      .word 15      # límite del ciclo
    val3:   .word 70
    val5:   .word 66
    val15:  .word 7066
.text
   # Su codigo aqui
```

Solución: A continuación se presenta una alternativa de código al problema planteado.

```
.data
    n:      .word 15      #  lmite del ciclo
    val3:   .word 70      # valor para multiplos de 3
    val5:   .word 66      # valor para multiplos de 5
    val15:  .word 7066    # valor para multiplos de 15

.text
    addi s0, zero, 1 # contador

    la t0, n
    lw s2, 0(t0) # s2 = n

    addi s3, zero, 3 # primer divisor
    addi s4, zero, 5 # segundo divisor
    addi s5, zero, 15 # tercer divisor

loop:
    blt s2, s0, end # si i > n -> termino

    # Divisible por 15
    rem t1, s0, s5 # calcular el resto (i % 15)
    beq t1, zero, both # si el resto es 0 salto a both

    # Divisible por 3
    rem t1, s0, s3 # calcular el resto (i % 3)
    beq t1, zero, div3 # si el resto es 0 salto a div3

    # Divisible por 5
    rem t1, s0, s4 # calcular el resto (i % 5)
    beq t1, zero, div5 # si el resto es 0 salto a div5

    # Imprimo el  nmero
    addi a0, s0, 0
    addi a7, zero, 1
    ecall
    jal zero, newline  # saltar a newline

both:
    la t0, val15
    lw a0, 0(t0) # a0 = 7066

    # imprimir
    addi a7, zero, 1
    ecall

    jal zero, newline # saltar a newline

div3:
    la t0, val3 # a0 = 70
    lw a0, 0(t0)

    # imprimir
    addi a7, zero, 1
    ecall

    jal zero, newline # saltar a newline

div5:
    la t0, val5 # a0 = 66
    lw a0, 0(t0)

    # imprimir
    addi a7, zero, 1
    ecall
```

```
newline:
    addi a0, zero, 10
    addi a7, zero, 11
    ecall

next:
    addi s0, s0, 1 # i = i + 1
    jal zero, loop

end:
    addi a7, zero, 10
    ecall
```

Pregunta 4: Explique el código (AP-S2-P1-2025-1)

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_x`:

```
.data
  x: .word 4
.text
  lw t0, x
  addi s0, zero, 1
  addi sp, sp, -4
  sw ra, 0(sp)
  jal ra, func_x
  lw ra, 0(sp)
  add a0, zero, t0
  addi a7, zero, 1
  ecall
  addi a7, zero, 10
  ecall
  func_x:
    beq t0, s0, end_func_x
    addi sp, sp, -8
    sw ra, 0(sp)
    sw t0, 4(sp)
    addi t0, t0, -1
    jal ra, func_x
    lw ra, 0(sp)
    lw t1, 4(sp)
    addi sp, sp, 8
    mul t0, t0, t1
    end_func_x:
      jalr zero, 0(ra)
```

Este fragmento representa el cómputo de una función $f(x)$. A partir de este:

1. (1.5 ptos.) Indique, con argumentos y en términos de x, lo que retorna la función $f(x)$. Por ejemplo, $f(x) = x + 1$.

   Solución: La función computa el factorial de x, *i.e.* $f(x) = x!$. Esto se evidencia en el hecho de que el valor de retorno, almacenado en el registro `t0`, es igual a $x \times f(x-1)$, con caso base $f(1) = 1$. De esta forma, se tiene que:

   $$f(x) = x \times f(x-1) = x \times (x-1) \times f(x-2) = \cdots = x \times (x-1) \times (x-2) \times \cdots \times 1 = x!$$

2. (1.5 ptos.) Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.

   Solución: El fragmento anterior **no respeta** la convención de llamadas de RISC-V. El motivo radica en el almacenamiento del argumento x y del valor de retorno de la subrutina `func_x`: si se siguiera la convención, se habría usado el registro `a0` tanto para almacenar x como para retornar $x!$. No obstante, se hace uso del registro `t0`, el que está pensando en el almacenamiento de valores temporales que pueden ser sobreescritos por llamados a subrutinas.

Pregunta 5: Elabore el código (AP-S3-P2-2025-1)

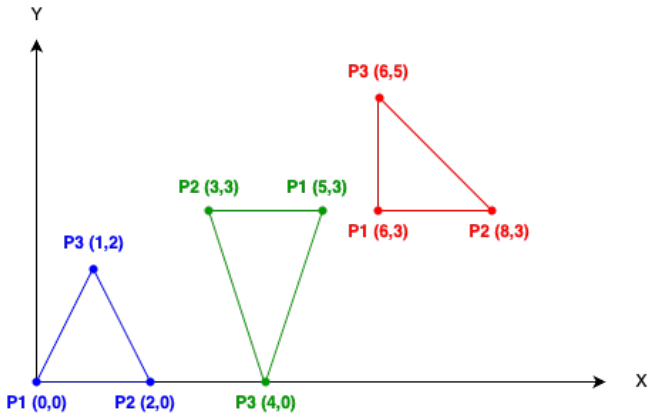
Elabore, utilizando el Assembly RISC-V, un programa que **calcule el área del triángulo que se forma a partir de tres puntos en un plano de dos dimensiones**. Las coordenadas de los puntos se guardarán individualmente en la memoria, como se muestra a continuación:

```
# Calcular el área de un triángulo que tiene un segmento paralelo al eje x
.data
  area:   .word 0
  b:      .word 0
  h:      .word 0
  P1:     .word 0, 0  # Coordenadas (X,Y) del punto P1
  P2:     .word 2, 0  # Coordenadas (X,Y) del punto P2
  P3:     .word 1, 2  # Coordenadas (X,Y) del punto P3
.text
  # Su código aquí. Guarde los resultados de base, altura y área en las variables correspondientes.
```

Puede asumir los siguientes supuestos:

- Los tres puntos **siempre** formarán un triángulo válido.
- Los tres puntos **siempre** se encontrarán en el primer cuadrante del plano cartesiano, *i.e.* $x \geq 0$ y $y \geq 0$ para todo punto (x, y).
- Los puntos `P1` y `P2` conformarán la base del triángulo y **siempre** será paralela al eje x.
- El cálculo del área se obtiene a partir de la fórmula $\frac{b \times h}{2}$, siendo b la extensión de la base y h la extensión de la altura del triángulo.

A continuación, tres ejemplos de triángulos válidos en el contexto de este problema:



IMPORTANTE: No es necesario que respete la convención en este ejercicio.

Solución: A continuación se presenta un código que cumple con lo solicitado:

```
# Calcular el área de un triángulo que tiene un segmento paralelo al eje x
.data
  area:    .word 0
  b:       .word 0
  h:       .word 0
  P1:      .word 0, 0  # Coordenadas (X,Y) del punto P1
  P2:      .word 2, 0  # Coordenadas (X,Y) del punto P2
  P3:      .word 1, 2  # Coordenadas (X,Y) del punto P3
.text
  init:
    addi s0, zero, -1
    addi s1, zero, 2
    la s2, b
    la s3, h
    la s4, area
    la t0, P1
    lw t2, 4(t0)       # t2 = P1.y
    lw t0, P1          # t0 = P1.x
    lw t1, P2          # t1 = P2.x
    la t3, P3
    lw t3, 4(t3)       # t3 = P3.y
  base_compute:
    sub t0, t0, t1
    bgt t0, zero, height_compute
    mul t0, t0, s0
  height_compute:
    sw t0, 0(s2)
    sub t2, t2, t3
    bgt t2, zero, area_compute
    mul t2, t2, s0
  area_compute:
    sw t2, 0(s3)
    mul t4, t0, t2
    div t4, t4, s1
    sw t4, 0(s4)
  end:
    addi a7, zero, 10
    ecall
```

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Guía de ejercicios: RISC-V (Subrutinas Y Convención)

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl), José Mendoza (jfmendoza@uc.cl)

Pregunta 1: Ejercicio programacion de subrutinas

Implemente en lenguaje ensamblador RISC-V dos subrutinas. Para este ejercicio, no es necesario seguir la convención de llamadas estrictamente, solo tienes que cuidar de respaldar `ra` para que no se pierda entre llamadas de subrutinas.

1. **Subrutina `multiplicar_por_dos`:** Recibe un número entero en `a0`, lo duplica (calcula $a0 \cdot 2$) y retorna el resultado en `a0`.
2. **Subrutina `calcular_perimetro`:** Recibe el ancho de un rectángulo en `a0` y el alto en `a1`. Debe calcular el perímetro total ($2 \cdot ancho + 2 \cdot alto$) llamando a la subrutina `multiplicar_por_dos` para cada lado, y retornar el resultado final en `a0`.

Se da como base el siguiente código:

```
.data
    ancho:      .word   5
    alto:       .word   3
.text
main:
    la   t0, ancho
    lw   a0, 0(t0)              # a0 = ancho (5)
    la   t0, alto
    lw   a1, 0(t0)              # a1 = alto (3)

    jal  ra, calcular_perimetro # Llamada a la función principal

    addi a7, zero, 10           # Terminar ejecución
    ecall

multiplicar_por_dos:
    # Completa el codigo aqui
    jalr zero, 0(ra)

calcular_perimetro:
    # Completa el codigo aqui
    jalr zero, 0(ra)
```

Solución:

```
multiplicar_por_dos:
    add  a0, a0, a0             # a0 = a0 * 2
    jalr zero, 0(ra)

calcular_perimetro:
    addi sp, sp, -4             # Reservar espacio en stack
    sw   ra, 0(sp)              # Guardar ra original

    addi t0, a1, 0              # t0 = alto
    jal  ra, multiplicar_por_dos # a0 = 2 * ancho

    addi t1, a0, 0              # t1 = 2 * ancho
    addi a0, t0, 0              # a0 = alto
    jal  ra, multiplicar_por_dos # a0 = 2 * alto

    add  a0, t1, a0             # a0 = (2 * ancho) + (2 * alto)

    lw   ra, 0(sp)              # Restaurar ra original
    addi sp, sp, 4              # Liberar stack
    jalr zero, 0(ra)
```

Pregunta 2: Explique el código [AP-S2-2025'2]

En el siguiente fragmento de código se realiza el llamado de una subrutina `func_n_m`:

```
.data
n:  .word 37
m:  .word 5
r:  .word 0
.text
main:
    addi sp, sp, -16
    sw ra, 12(sp)
    la t0, n
    lw a0, 0(t0)
    la t1, m
    lw a1, 0(t1)
    jal ra, func_n_m
    la t2, r
    sw a0, 0(t2)
    lw ra, 12(sp)
    addi sp, sp, 16
    addi a7, zero, 10
    ecall

func_n_m:
    beq a0, zero, ccc
    addi sp, sp, -16
    sw ra, 12(sp)
    sw a0, 8(sp)
    addi a0, a0, -1
    jal ra, func_n_m
    lw t0, 8(sp)
    lw ra, 12(sp)
    addi sp, sp, 16
    add a0, a0, a1
    jalr zero, 0(ra)

ccc:
    addi a0, zero, 0
    jalr zero, 0(ra)
```

Este fragmento representa el cómputo de una función $f(n, m)$. A partir de este:

1. **(1.5 ptos.)** Indique, con argumentos y en términos de n y m, lo que retorna la función $f(n, m)$.
   Por ejemplo, $f(n, m) = n + m$. Se otorgan 0.75 ptos. por la correctitud de la descripción del retorno y 0.75 ptos. por justificación.
2. **(1.5 ptos.)** Indique, con argumentos, si el fragmento anterior respeta o no la convención de llamadas de RISC-V.
   Se otorgan 0.75 ptos. si indica de forma correcta si se respeta o no la convención y 0.75 ptos. por entregar una justificación válida respecto a su respuesta.

Solución:

1. La función implementa la multiplicación mediante sumas sucesivas, por lo que:

$$f(n, m) = n \times m$$

   En el código, `a0` contiene el argumento n y actúa como contador decreciente en cada llamado recursivo. El caso base ocurre cuando `a0` $= 0$, retornando 0. En cada retorno desde la recursión, se suma `a1` (que contiene m) al valor acumulado en `a0`, de modo que se realizan n sumas del valor m. El resultado final ($n \times m$) queda almacenado en `a0`, y luego se guarda en `r`.

2. El fragmento anterior **NO** respeta la convención de llamadas de RISC-V. Aunque el código ejecuta correctamente la multiplicación y maneja el stack sin errores de ejecución, desde el punto de vista de la convención formal no cumple con las reglas establecidas. En particular:

   - Los registros `a0` y `a1` son *caller-saved*. En el programa principal (`main`) se realiza el llamado a `func_n_m` sin respaldar previamente ninguno de estos registros, a pesar de que ambos pueden ser modificados por la subrutina.
   - Dentro de la subrutina `func_n_m`, el registro `a1` tampoco es respaldado antes del llamado recursivo, incumpliendo nuevamente el requisito de *caller-saved*, ya que la función se llama a sí misma y puede alterar sus propios argumentos.
   - El registro `t0`, que es de tipo *caller-saved*, se utiliza dentro de la función después del retorno recursivo sin haber sido respaldado antes del llamado, lo que también infringe la convención.

Pregunta 3: Ejercicio de Programación y Convención de Llamadas

Considere un sistema embebido que recibe bloques de datos de un sensor. Se le solicita implementar en lenguaje ensamblador RISC-V dos subrutinas que permitan filtrar y promediar estos datos.

1. **Subrutina `filtrar_valor`:** Recibe un entero por valor en el registro `a0`. Si el entero es negativo, lo transforma en 0. Si es positivo, lo mantiene igual. Retorna el valor filtrado en `a0`.
2. **Subrutina `procesar_buffer`:** Recibe en `a0` la dirección de inicio de un arreglo de enteros de 32 bits y en `a1` el tamaño N del arreglo. Esta función debe recorrer el arreglo, aplicar la función `filtrar_valor` a cada elemento, modificar el arreglo original en memoria con los nuevos valores filtrados, y finalmente **retornar la suma** de todos los valores ya filtrados en `a0`.

Desarrolle el código en ensamblador RISC-V para ambas subrutinas, respetando la convención de llamados para el respaldo de registros en el *stack*. Se da como base el siguiente código:

```
.data
# Arreglo de prueba con valores positivos y negativos
buffer:     .word   5, -3, 12, 0, -8, 7
tamaño:     .word   6

.text

main:
    # Cargar dirección del buffer y el tamaño
    la   a0, buffer             # Carga dirección base de prueba
    la   t0, tamaño
    lw   a1, 0(t0)              # a1 = 6

    jal  ra, procesar_buffer    # Llamada a la función principal del ejercicio

    # Al volver, a0 contiene la suma total. dirección
    # Terminar ejecución
    addi a7, zero, 10           # Código de salida
    ecall


filtrar_valor:
    # Completa el codigo aqui
    jalr zero, 0(ra)

procesar_buffer:
    # Completa el codigo aqui
    jalr zero, 0(ra)
```

Solución:

```
filtrar_valor:
    blt  a0, zero, es_negativo
    jalr zero, 0(ra)

    es_negativo:
        addi a0, zero, 0
        jalr zero, 0(ra)

procesar_buffer:
    addi sp, sp, -24
    sw   ra, 20(sp)             # Guardar dirección de retorno original
    sw   s0, 16(sp)             # s0 = dirección del elemento actual
    sw   s1, 12(sp)             # s1 = contador N (elementos restantes)
    sw   s2, 8(sp)              # s2 = acumulador de la suma

    addi s0, a0, 0              # s0 = dirección del primer elemento
    addi s1, a1, 0              # s1 = N
    addi s2, zero, 0            # s2 = suma inicializada en 0

    loop:
        beq  s1, zero, fin         # Si N == 0, salir del ciclo

        lw   a0, 0(s0)              # Cargar elemento actual de la memoria

        jal  ra, filtrar_valor      # Llamar a la subrutina, resultado en a0

        sw   a0, 0(s0)              # Guardar el valor filtrado en la memoria

        add  s2, s2, a0             # Acumular en la suma

        addi s0, s0, 4              # Avanzar al siguiente entero (4 bytes)
        addi s1, s1, -1             # Decrementar el contador N

        jal  zero, loop

    fin:
        addi a0, s2, 0              # Colocar la suma total en a0

        lw   ra, 20(sp)
        lw   s0, 16(sp)
        lw   s1, 12(sp)
        lw   s2, 8(sp)
        addi sp, sp, 24

        jalr zero, 0(ra)
```

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2024)

Interrogación 2 - RISC-V

Instrucciones

Lea atentamente el enunciado. Tendrá hasta las 21:00 del 29 de Mayo para subirlo a Canvas. Cualquier duda que tenga, debe realizarla a través de las *issues* del repositorio del curso usando el *label* "`Actividad RISC-V`".

Pregunta 4: Batalla Pokémon (8 ptos.)

El Dr. Yadran quiere completar una Pokédex regional de Santiago, por lo que requiere de tu ayuda para completar su objetivo. Para ello, deberás simular el combate de los Pokémon^{TM} *starters* de Santiago haciendo uso del lenguaje de ensamblado RISC-V en el emulador RARS.

Cada Pokémon estará ubicado en la sección de `.data` y tendrá los siguientes valores almacenados de manera **consecutiva**.

Offset	Nombre	Descripción
0	**ID**	Número del Pokémon en la Pokédex
4	**Tipo**	Puede ser tipo: planta (0), fuego (1) o agua (2)
8	**Nivel**	Nivel del Pokémon
12	**EXP**	Puntos de experiencia. El Pokémon los obtiene al ganar peleas
16	**Estado**	Estado de salud del Pokémon. Saludable (0), envenenado (1), dormido (2) o desmayado (3)
20	**HP**	*Health Points*. Cantidad de puntos de salud que tiene el Pokémon
24	**HP total**	Cantidad máxima de HP que puede tener el Pokémon
28	**Ataque**	Puntos de ataque físico del Pokémon
32	**Defensa**	Puntos de defensa físico del Pokémon
36	**Ataque Especial**	Puntos de ataque especial del Pokémon
40	**Defensa Especial**	Puntos de defensa especial del Pokémon
44	**Velocidad**	Puntos de velocidad del Pokémon

La batalla va a consistir en una de enfrentamientos entre dos Pokémon. Se enfrentarán hasta que alguno de ellos llegue al **nivel** 40. Después de cada enfrentamiento, todos los Pokémon recuperarán completamente sus puntos de salud. De esta manera, el flujo de batalla general es el siguiente:

1. El Pokémon más rápido es el primero en atacar a su contrincante. Esto ocurre solo si **no está dormido**.
2. Luego, es el turno del Pokémon más lento para atacar. Esto ocurre solo si **no está dormido**.
3. Si alguno de los Pokémon pierde todos sus puntos de vida después de ser atacado, la pelea finaliza y el ganador se lleva los **puntos de experiencia**. El Pokémon que pierde pasa a estado **desmayado**.
4. Si ninguno de los Pokémon se desmayó en el turno, se revisa lo siguiente:
   - Si un Pokémon se encuentra **envenenado**, pierde puntos de salud después de la ronda.
   - Si un Pokémon está **dormido**, tiene 50 % de probabilidades de despertar.
5. Al finalizar el turno, hay un 10 % de probabilidades que un Pokémon se **envenene** y un 10 % de probabilidades de que se **duerma**.
6. El flujo anterior se repite hasta que haya un ganador.

Este flujo **ya se encuentra implementado** en un archivo base adjunto a este enunciado, **¡no debe programarlo!** Su tarea consistirá en **completar** el código de un conjunto de subrutinas, las que son descritas en las siguientes secciones. Adicionalmente, en este mismo archivo contará con subrutinas complementarias **ya implementadas** para facilitar su desarrollo.

1-. Ataque: Subrutina `attack`

Para computar el ataque de un Pokémon en un turno, debe hacer uso de la siguiente fórmula:

$$\texttt{Poder Ataque} = \texttt{Ataque físico (atacante)} - \texttt{Defensa fisica (defensor)} + \texttt{Ponderador tipo} + 4 \times \texttt{Ataque crítico}$$

El `Ponderador tipo` corresponde a un ponderador que indica si el ataque es más efectivo (o no) dado el tipo de los Pokémon atacante y defensor.

Pokémon Atacante	Pokémon Defensor	Ponderador
Planta (0)	Planta (0)	0
Planta (0)	Fuego (1)	-3
Planta (0)	Agua (2)	3
Fuego (1)	Planta (0)	3
Fuego (1)	Fuego (1)	0
Fuego (1)	Agua (2)	-3
Agua (2)	Planta (0)	-3
Agua (2)	Fuego (1)	3
Agua (2)	Agua (2)	0

Para lograr lo anterior, debe implementar la subrutina `attack`. Esta debe tomar como argumentos las direcciones del primer y segundo Pokémon a través de los registros `a0` y `a1` , respectivamente, y hacer lo siguiente:

1. Primero, debe verificar si el Pokémon se encuentra **dormido**. Si lo está, entonces se retorna y pierde su turno de ataque.
2. Si no esta dormido, entonces puede atacar. Para calcular el ataque, debe obtener el ponderador por tipo; el ataque físico del atacante; la defensa física del defensor; y determinar si el ataque es crítico.
3. Para obtener el ponderador por tipo, se debe llamar a la subrutina `_type_boost`. Esta toma como argumentos el valor de la dirección base del Pokémon atacante en el registro `a0` , y en el registro `a1` el valor de la dirección base del Pokémon defensor. La subrutina retorna el ponderador (-3, 0 o 3 según sea el caso) en el registro `a0` .
4. Para saber si el ataque es crítico, se debe llamar a la subrutina `_critical_damage`. Esta toma como argumento la dirección base del Pokémon en el registro `a0` y hace lo siguiente: Retorna 1 con una probabilidad del 10 %, y 0 en cualquier otro caso. El valor de retorno es almacenado en `a0` . Si el valor de retorno es 1, entonces el daño es crítico y el daño del Pokémon atacante aumenta en 4 puntos. Adicionalmente imprime en consola el nombre del Pokémon , indicando si realizará un daño crítico o no.
5. Luego de obtener los valores anteriores, computa el valor de ataque según la fórmula antes descrita. Si el valor calculado es menor a 0, entonces el ataque será igual a 0.
6. Se restan los puntos calculados al **HP** del Pokémon defensor.
7. Si el **HP** del Pokémon defensor es mayor a 0, se debe retornar.
8. Si el **HP** del Pokémon defensor es menor o igual a 0, el valor de **HP** se deja en 0 y se debe llamar a la subrutina `_faint`. Esta toma como argumento la dirección base del Pokémon en el registro `a0` e imprime en consola su nombre, indicando que se ha desmayado.
9. Retorna a donde fue llamada.

2. Turno ataque: Subrutina `turn_attack`

Para llevar acabo el ataque por turnos, se debe implementar la subrutina `turn_attack`. Esta debe tomar como argumentos las direcciones del primer y segundo Pokémon a través de los registros `a0` y `a1` , respectivamente, y hacer lo siguiente:

1. Primero se debe decidir el orden en que deben atacar. El Pokémon con mayor **velocidad** es el que debe golpear primero.
2. El Pokémon con mayor velocidad ataca primero. Para atacar se debe llamar a la subrutina `attack`. Se debe guardar la dirección base del Pokémon más rápido en el registro `a0` y la dirección base del Pokémon más lento se debe guardar en el registro `a1` .
3. Si al retornar de `attack` el Pokémon que fue atacado tiene **HP** igual a 0, se debe retornar (pierde el turno de atacar).
4. Ahora es el turno del Pokémon con menor velocidad. Para atacar se debe llamar a la subrutina `attack`. Se debe guardar la dirección base del Pokémon más lento en el registro `a0` y la dirección base del Pokémon más rápido se debe guardar en el registro `a1` .
5. Retorna a donde fue llamada.

3. Experiencia: Subrutina `experience`

Cada vez que un Pokémon gana una batalla, recibe puntos de experiencia (**EXP**). La experiencia que gana un Pokémon depende de: el nivel que se encuentra; su ataque físico y su ataque especial. La fórmula con la que se calcula la **experiencia** recibida por ganar una batalla es la siguiente:

$$\texttt{Experiencia ganada} = \texttt{Nivel} + \texttt{Ataque físico} + \texttt{Ataque especial}$$

Para esto, se debe implementar la subrutina `experience`. Esta debe tomar como argumento el valor de la dirección base del Pokémon a través del registro `a0` y hacer lo siguiente:

1. Calcular la experiencia que gana un Pokémon al ganar una batalla a través de la fórmula antes descrita.
2. Sumar el valor calculado a la **experiencia** que el Pokémon ya posee.
3. Si la **experiencia** total es menor a 100, se debe retornar.
4. Si la **experiencia** total es mayor o igual a 100, se debe subir de nivel. Antes de hacerlo, debe guardar como **experiencia** la resta entre el valor computado y 100. Esto es importante, ya que este parámetro siempre debe estar entre 0 y 99.
5. Para subir el nivel del Pokémon , se debe llamar a la subrutina `_level_up`. Esta toma como argumento la dirección base del Pokémon en el registro `a0` y actualiza las estadísticas del mismo: aumenta el nivel, ataque físico, ataque especial, defensa física, defensa especial, velocidad y, por último, llama a la subrutina `evolve` descrita en la siguiente sección. Adicionalmente, imprime en consola su nombre indicando que ha subido de nivel.
6. Retorna a donde fue llamada.

4. Evolución: Subrutina `evolve`

Dada la naturaleza de cada Pokémon , al hacerse más fuertes por su experiencia pueden transformarse en una nueva especie más poderosa, lo que se conoce como **evolución**. Por lo anterior, cada vez que un Pokémon sube de nivel, se verifica si este debe evolucionar. Para esto, se debe implementar la subrutina `evolve`. Esta debe tomar como argumento el valor de la dirección base del Pokémon a través del registro `a0` y hacer lo siguiente:

1. Debe buscar en `evolutions`, correspondiente a una lista de tripletas en `.data`, la tripleta del Pokémon recibido como argumento. Cada tripleta tiene la siguiente estructura: ID del Pokémon, ID del Pokémon al que evoluciona, y **nivel** en el que evoluciona.
2. Si **no** se encuentra una tripleta con el primer elemento igual al **ID** del Pokémon recibido como argumento, se debe retornar.
3. Si se encuentra una tripleta con el primer elemento igual al **ID** del Pokémon recibido como argumento, se debe que verificar si tiene el nivel necesario para evolucionar o no.
4. Para verificar si tiene el nivel necesario para evolucionar, este debe tener un nivel mayor o igual al tercer elemento de la tripleta. Si es menor, se debe retornar.

5. Si el Pokémon cumple con los requisitos para evolucionar, se debe cambiar su **ID** al segundo valor contenido en la tripleta y llamar a la subrutina `_print_evolve_text`. Esta toma como argumento la dirección base del Pokémon en el registro `a0` e imprime en consola su nombre, indicando que evolucionó.
6. Retorna a donde fue llamada.

Bonus opcional - 5. Estado: Envenenado

Al final de cada turno, si un Pokémon tiene como estado de salud **envenenado**, su vida (**HP**) debe decrementar en 2 puntos. Para esto, se debe implementar la subrutina `poison_damage`. Esta debe tomar como argumento el valor de la dirección base del Pokémon a través del registro `a0` y hacer lo siguiente:

1. Primero debe verificar si el Pokémon tiene como estado de salud **envenenado**. Si no, debe retornar.
2. Si el Pokémon esta **envenenado**, se debe restar 2 puntos a su salud **HP**.
3. Se debe llamar a la subrutina `_print_poison_text`. Esta toma como argumento la dirección base del Pokémon en el registro `a0` e imprime en consola el nombre del Pokémon **envenenado**.
4. Si los puntos de vida disminuyen a un valor menor o igual a 0, se debe guardar un 0 en su **HP** y llamar a la subrutina `_faint`. Esta toma como argumento la dirección base del Pokémon en el registro `a0` e imprime en consola su nombre, indicando que se desmayó.
5. Retorna a donde fue llamada.

Bonus opcional - 6. Estado: Dormido

Al final de cada turno, si un Pokémon tiene como estado de salud **dormido**, tiene un 50 % de probabilidades de despertar.

Para esto, se debe implementar la subrutina `wake_up`. Esta debe tomar como argumento el valor de la dirección base del Pokémon a través del registro `a0` y hacer lo siguiente:

1. Primero, debe verificar si el Pokémon tiene como estado de salud **dormido**. Si no, debe retornar.
2. Llamar a la subrutina `_rand_number`. Esta recibe como argumento un número a través del registro `a0` , y retorna un valor en el rango [0,`a0` [. Por ejemplo, si `a0` = 10, entonces retorna un número entre 0 y 9. El valor retornado se almacena en el registro `a0` .
3. Dado el número generado, se decide con un 50 % de probabilidad si el Pokémon despierta o no. Si sigue **dormido**, hay que retornar.
4. Si el Pokémon despierta, se debe cambiar su **estado** a **saludable** y llamar a la subrutina `_print_wake_up_text`. Esta toma como argumento la dirección base del Pokémon en el registro `a0` e imprime en consola el nombre su nombre, indicando que despertó.
5. Retorna a donde fue llamada.

Importante - Convención RISC-V

En esta actividad, no se evaluará que respete la convención RISC-V. No obstante, si no se asegura del respaldo de los registros en las subrutinas a completar, **podría sobreescribir valores que impliquen una implementación incorrecta**. Por lo tanto, trate de respetar la convención revisando el código base entregado y asegurando el respaldo de los registros cuando corresponda.

Puntajes

El puntaje se otorga por el funcionamiento correcto de cada subrutina. Si presenta como máximo un error de implementación, se otorga la mitad del puntaje. En otro caso, se otorgan **0 ptos.**
El puntaje de bonificación, en caso de obtenerlo, se sumará directamente al total de la I2.

(a) (1.5 ptos.) Implementa la subrutina `attack` según lo especificado en Ataque.

Solución:

```
attack:                                 # a0: direccion del pokemon a atacante. a1: direccion del
    pokemon a defensor.
        addi sp, sp, -16
        sw ra, 0(sp)
        sw a0, 4(sp)
        sw a1, 8(sp)
        sw s0, 12(sp)

        lw t0, 16(a0)
        addi t1, zero, 2
        bne t0, t1, no_dormido
        addi sp, sp, 16
        jalr zero, 0(ra)

        no_dormido:
        jal ra, _type_boost
        add s0, zero, a0
        lw a0, 4(sp)
        jal ra, _critical_damage
        addi t0, zero, 4
        mul t0, a0, t0
        add s0, s0, t0

        lw a0, 4(sp)
        lw a1, 8(sp)
        lw t0, 28(a0) # Se guarda en t0 el ataque
        lw t1, 32(a1) # Se guarda en t1 la defensa del segundo
        sub t0, t0, t1 # Ataque - Defensa
        add s0, s0, t0 # (Ataque - Defensa) + critico + ponderador
        bgt s0, zero, ataque_positivo
        add s0, zero, zero

        ataque_positivo:
        lw t0, 20(a1) # Se guarda en t1 el HP del segundo
        sub t0, t0, s0
        bgt t0, zero, seguir_ataque
        sw zero, 20(a1)
        lw a0, 8(sp)
```

```
        jal ra, _faint
        lw a0, 4(sp)
        lw ra, 0(sp)
        lw s0, 12(sp)
        addi sp, sp, 16
        jalr zero, 0(ra)

        seguir_ataque:
        sw t0, 20(a1) # t1 se guarda en el HP del segundo
        lw ra, 0(sp)
        lw s0, 12(sp)
        addi sp, sp, 16
        jalr zero, 0(ra)
```

1. (0.25 puntos) El *output* del test `test_01_attack.asm` es corecto.
2. (0.25 puntos) El *output* del test `test_02_attack.asm` es corecto.
3. (0.25 puntos) El *output* del test `test_03_attack.asm` es corecto.
4. (0.25 puntos) El *output* del test `test_04_attack.asm` es corecto.
5. (0.5 puntos) El *output* del test `test_05_attack_sp.asm` es corecto.

(b) (1.5 ptos.) Implementa la subrutina `turn_attack` según lo especificado en Turno Ataque.

Solución:

```
turn_attack:                    # a0: direccion del primer pokemon. a1: direccion del segundo
    pokemon.
        addi sp, sp, -12
        sw ra, 0(sp)
        sw a0, 4(sp)
        sw a1, 8(sp)

        lw t0, 44(a0)
        lw t1, 44(a1)

        blt t0, t1, pokemon2_parte

        pokemon1_parte:
                jal ra, attack
                lw a0, 8(sp)
                lw a1, 4(sp)
                lw t0, 20(a0)
                beq t0, zero, retornar_turno_ataque
                jal ra, attack
                beq zero, zero, retornar_turno_ataque

        pokemon2_parte:
                lw a0, 8(sp)
                lw a1, 4(sp)
                jal ra, attack
                lw a0, 4(sp)
                lw a1, 8(sp)
                lw t0, 20(a0)
                beq t0, zero, retornar_turno_ataque
                jal ra, attack
```

```
        retornar_turno_ataque:
        lw ra, 0(sp)
        lw a0, 4(sp)
        lw a1, 8(sp)
        addi sp, sp, 12
        jalr zero, 0(ra)
```

1. (0.25 puntos) El *output* del test `test_06_turn_attack.asm` es correcto.
2. (0.25 puntos) El *output* del test `test_07_turn_attack.asm` es correcto.
3. (0.25 puntos) El *output* del test `test_08_turn_attack.asm` es correcto.
4. (0.25 puntos) El *output* del test `test_09_turn_attack.asm` es correcto.
5. (0.5 puntos) El *output* del test `test_10_turn_attack_sp.asm` es correcto.

(c) (1.5 ptos.) Implementa la subrutina `evolve` según lo especificado en Evolución.

Solución:

```
evolve:                                 # a0: direccion del pokemon a evolucionar.
        addi sp, sp, -8
        sw ra, 0(sp)
        sw a0, 4(sp)

        lw t0, 0(a0)    # ID
        lw t1, 8(a0)    # Nivel

        la t2, evolutions
        loop_evolve:
        lw t3, 0(t2)
        bgt t3, t0, retornar_evolve
        beq t0, t3, evolucionar
        addi t2, t2, 12
        beq zero, zero, loop_evolve

        evolucionar:
        lw t3, 4(t2) # Next ID
        lw t4, 8(t2) # Lvl
        blt t1, t4, retornar_evolve
        sw t3, 0(a0)
        jal ra, _print_evolve_text

        retornar_evolve:
        lw ra, 0(sp)
        lw a0, 4(sp)
        addi sp, sp, 8
        jalr zero, 0(ra)
```

1. (0.25 puntos) El *output* del test `test_11_experience.asm` es correcto.
2. (0.25 puntos) El *output* del test `test_12_experience.asm` es correcto.
3. (0.25 puntos) El *output* del test `test_13_experience.asm` es correcto.
4. (0.25 puntos) El *output* del test `test_14_experience.asm` es correcto.

5. (0.5 puntos) El *output* del test `test_15_experience_sp.asm` es correcto.

(d) (1.5 ptos.) Implementa la subrutina `experience` según lo especificado en Experiencia.

Solución:

```
experience:                             # a0: direccion del pokemon ganador de experiencia.
        lw t0, 8(a0)    # Nivel del pokemon a0
        lw t1, 28(a0)   # Ataque del pokemon a0
        lw t2, 36(a0)   # Ataque especial del pokemon a0
        add t0, t0, t1
        add t0, t0, t2
        lw t1, 12(a0)   # Experiencia del pokemon a0
        add t0, t0, t1  # Experiencia del actual pokemon a0
        sw t0, 12(a0)   # Experiencia del pokemon a0
        addi t1, zero, 100
        blt t0, t1, seguir_experience
        addi t0, t0, -100
        sw t0, 12(a0)   # Experiencia del pokemon a0
        addi sp, sp, -4
        sw ra, 0(sp)
        jal ra, _level_up
        lw ra, 0(sp)
        addi sp, sp, 4
        seguir_experience:
        jalr zero, 0(ra)
```

1. (0.25 puntos) El *output* del test `test_16_evolve.asm` es correcto.
2. (0.25 puntos) El *output* del test `test_17_evolve.asm` es correcto.
3. (0.25 puntos) El *output* del test `test_18_evolve.asm` es correcto.
4. (0.25 puntos) El *output* del test `test_19_evolve.asm` es correcto.
5. (0.5 puntos) El *output* del test `test_20_evolve_sp.asm` es correcto.

(e) (1 pto.) **Bonus:** Implementa la subrutina `poison_damage` según lo especificado en Estado: Envenenado.

Solución:

```
poison_damage:                  # a0: direccion del pokemon a verificar envenenamiento.
        addi sp, sp, -8
        sw ra, 0(sp)
        sw a0, 4(sp)

        # Verificar si esta envenenado
        lw t0, 16(a0)
        addi t1, zero, 1
        bne t0, t1, retornar_poison

        # Traer HP y restar 2
        lw t0, 20(a0)
        addi t0, t0, -2
```

```
        bgt t0, zero, seguir_poison
        # Se murio :(
        sw zero, 20(a0)
        jal ra, _print_poison_text
        lw a0, 4(sp)
        jal ra, _faint
        beq zero, zero, retornar_poison

        seguir_poison:
        sw t0, 20(a0)
        jal ra, _print_poison_text
        retornar_poison:
        lw ra, 0(sp)
        addi sp, sp, 8
        jalr zero, 0(ra)
```

1. (0.2 puntos) El *output* del test `test_21_poison.asm` es correcto.
2. (0.2 puntos) El *output* del test `test_22_poison.asm` es correcto.
3. (0.2 puntos) El *output* del test `test_23_poison.asm` es correcto.
4. (0.2 puntos) El *output* del test `test_24_poison.asm` es correcto.
5. (0.2 puntos) El *output* del test `test_25_poison_sp.asm` es correcto.

(f) (1 pto.) **Bonus:** Implementa la subrutina `wake_up` según lo especificado en Estado: Dormido.

Solución:

```
wake_up:                                # a0: direccion del pokemon a despertar.
        addi sp, sp, -8
        sw ra, 0(sp)
        sw a0, 4(sp)
        lw t0, 16(a0)

        addi t1, zero, 2
        bne t0, t1, retornar_wake_up

        addi a0, zero, 10
        jal ra, _rand_number
        add t0, zero, zero
        addi t1, zero, 5
        bge a0, t1, retornar_wake_up
        lw a0, 4(sp)
        add t0, zero, zero
        sw t0, 16(a0)
        jal ra, _print_wake_up_text
        retornar_wake_up:
        lw ra, 0(sp)
        addi sp, sp, 8
        jalr zero, 0(ra)
```

1. (0.2 puntos) El *output* del test `test_26_sleep.asm` es correcto.
2. (0.2 puntos) El *output* del test `test_27_sleep.asm` es correcto.
3. (0.2 puntos) El *output* del test `test_28_sleep.asm` es correcto.

4. (0.2 puntos) El *output* del test `test_29_sleep.asm` es corecto.

5. (0.2 puntos) El *output* del test `test_30_sleep_sp.asm` es corecto.



DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2025)

Interrogación 2

Pauta de evaluación

Pregunta 1: Arquitecturas de Computadores (6 ptos.)

(a) (1.5 ptos.) Un programa, desarrollado en un lenguaje de programación específico, es compilado en dos computadores distintos: uno con ISA RISC, y otro con ISA CISC. ¿Cuál de las dos compilaciones resultará en un código de máquina de mayor tamaño, *i.e.* con mayor cantidad de instrucciones? Justifique su respuesta.

> **Solución:** El programa para la ISA RISC tendrá un tamaño de código de máquina mayor, ya que RISC define *sets* de instrucciones muy simples que redundan en una mayor cantidad de instrucciones en comparación a CISC para hacer una tarea específica (por ejemplo, multiplicaciones con sumas sucesivas en contraste con una instrucción directa que realice dicho cómputo). Se otorgan **0.75 ptos.** por responder correctamente y **0.75 ptos.** por justificación.

(b) (1.5 ptos.) Suponga que se modifica el computador básico estudiado en clases para que funcione con una única unidad de memoria, *i.e.* que su microarquitectura siga el modelo Von Neumann. ¿Implicaría esto cambios en su ISA? Justifique.

> **Solución:** No necesariamente. La ISA especifica la arquitectura del *set* de instrucciones que soporta el computador, por lo que la microarquitectura debe adaptarse a ella para su ejecución. El cambio del paradigma Harvard a Von Neumann implica que la microarquitectura debe modificarse para seguir soportando la ISA, y no al revés. Se otorgan **0.75 ptos.** por responder correctamente, y **0.75 ptos.** por justificación.

(c) (1.5 ptos.) Suponga que se le solicita revisar la arquitectura del computador básico estudiado en clases y, en caso de ser necesario, modificar su ISA para **reducir su consumo energético**, puesto que busca ser adaptada para dispositivos móviles. Indique si es necesario realizar cambios y de qué tipo; en caso contrario, explique por qué.

Solución: No es necesario realizar cambios. El motivo es que la ISA de tipo RISC ya de por sí es más óptima en términos de consumo energético respecto a CISC, dado que esta última soporta instrucciones más complejas con más ciclos de ejecución, implicando un mayor uso de energía por instrucción. Se otorgan **0.75 ptos.** por responder correctamente, y **0.75 ptos.** por justificación.

(d) (1.5 ptos.) Una arquitectura de tipo *Register-Memory* permite usar datos de la memoria como operandos de forma directa. En una arquitectura de tipo *Register-Register*, en cambio, es necesario que los datos se almacenen **antes** en los registros para poder operar con ellos. Indique, a partir de esta categorización y de forma argumentada, el tipo de arquitectura que presenta el computador básico estudiado en clases.

Solución: La arquitectura del computador básico es de tipo ***Register-Memory*** dado que la conexión entre la salida de la memoria de datos se conecta directamente con el multiplexor del operando `B` de la ALU. Esto permite la ejecución de instrucciones como `ADD A,(Dir)`, donde se usan valores de memoria como operandos de la ALU en un solo ciclo. Se otorgan **0.75 ptos.** por responder correctamente, y **0.75 ptos.** por justificación.

Pregunta 2: Comunicación con dispositivos I/O - Parte 1 (6 ptos.)

(a) (1.5 ptos.) En el pasado, la comunicación con dispositivos I/O se diseñó mediante mapeo a puertos como alternativa al mapeo a memoria, debido a una desventaja que este último presentaba en ese entonces. Hoy en día, esa limitación ha dejado de ser relevante, por lo que el mapeo a memoria es el que predomina en arquitecturas modernas. Explique cuál era la desventaja que tenía el mapeo a memoria en ese contexto, y por qué en la actualidad ya no representa un problema.

Solución: La desventaja que tenía el mapeo a memoria consistía en la *memory barrier*, *i.e.* la limitación del espacio direccionable por la reserva significativa de direcciones. Como en el pasado las memorias RAM tenían un tamaño reducido, esta pérdida era significativa e implicaba la imposibilidad de usar variables normalmente por la baja cantidad de direcciones disponibles. Hoy en día, en cambio, las memorias RAM poseen un tamaño lo suficientemente grande para que este problema sea inexistente. Se otorgan **0.75 ptos.** por responder correctamente, y **0.75 ptos.** por justificación.

(b) (1.5 ptos.) Explique el uso e importancia de la señal IRQ enviada por el controlador de interrupciones a la CPU.

Solución: La señal IRQ (*Interrupt Request*) es utilizada para comunicarle a la CPU que existe, al menos, un dispositivo I/O que requiere su atención. El controlador de interrupciones se encarga de su envío en caso de uno de los dispositivos de la arquitectura haya gatillado una interrupción. Es importante ya que es la que permite que inicie el proceso de interrupción, que termina en la ejecución de la ISR (*Interrupt Service Routine*) del dispositivo. Se otorgan **0.75 ptos.** por responder correctamente, y **0.75 ptos.** por justificación.

(c) (1.5 ptos.) Entregue dos ejemplos de interrupciones gatilladas por la CPU y no por dispositivos I/O, indicando su tipo (excepción o *trap*) y explicando en qué consisten. Ambos ejemplos pueden ser de un mismo tipo.

Solución: A continuación, se listan dos ejemplos:

- ***Stack overflow*:** Es una **excepción** que se gatilla cuando el tamaño de la memoria *stack* supera el límite permitido, generando posibles errores de sobreescritura en variables.
- ***Environment call*:** Es una ***trap*** que permite cederle el control al sistema operativo para que realice una acción, como imprimir en consola o terminar un proceso.

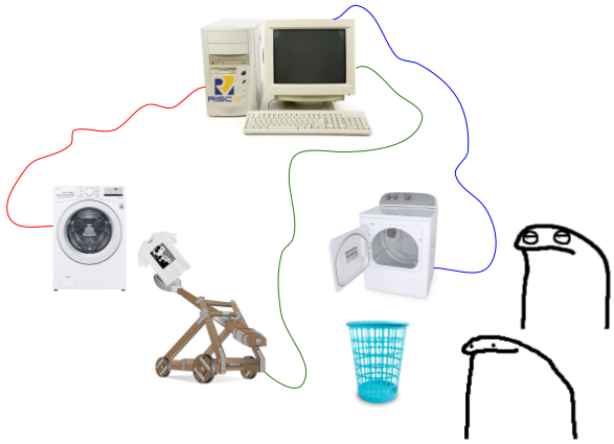
Se otorgan **0.75 ptos.** por cada ejemplo correctamente descrito. Se otorga la mitad del puntaje en un ejemplo si su descripción no es correcta, o bien su tipo no es consistente con el descrito.

(d) (1.5 ptos.) Suponga que, al computador básico estudiado en clases, se le añade todo el soporte en *hardware* necesario para comunicarse con dispositivos I/O mapeados en memoria a través de interrupciones. Además, uno de los dispositivos mapeados corresponde a un *pendrive*, del cual se desea copiar información a la RAM. ¿Es estrictamente necesario incluir *hardware* adicional para cumplir con esta funcionalidad? Justifique su respuesta.

Solución: En estricto rigor no es necesario. Si bien lo óptimo es contar con un controlador de DMA (*Direct Memory Access*) para orquestar la transferencia entre dispositivos y la memoria RAM, en la práctica se puede hacer uso de *Programmed* I/O para llevarla a cabo, *i.e.* usar los registros de CPU como puente para transferir los datos. Si bien esto en la práctica no se realiza por la cantidad de ciclos de CPU perdidos, se puede implementar como alternativa en caso de querer un *hardware* más sencillo. Se otorgan **0.75 ptos.** por responder correctamente, y **0.75 ptos.** por justificación.

Pregunta 3: Comunicación con dispositivos I/O - Parte 2 (6 ptos.)

Aburrido/a de los quehaceres del hogar, decide automatizar el lavado y secado de su ropa a través de un sistema centralizado que hace uso de la ISA RISC-V y de tres dispositivos: una lavadora (`washer`), una secadora (`dryer`) y una catapulta (`catapult`). A continuación, una ilustración de su innovadora idea:



Para llevar a cabo la comunicación entre la CPU y los dispositivos, los registros de 32 bits de estos últimos son mapeados en memoria desde la dirección 0xFEE1DEAD:

Offset	Nombre	Descripción
0x00	`washer_status`	Registro de estado de la lavadora
0x04	`washer_command`	Registro de comando de la lavadora
0x08	`washer_water_level`	Configuración de nivel de agua de la lavadora
0x0C	`dryer_status`	Registro de estado de la secadora
0x10	`dryer_command`	Registro de comando de la secadora
0x14	`dryer_type`	Configuración de tipo de ropa a secar
0x18	`catapult_status`	Registro de estado de la catapulta
0x1C	`catapult_command`	Registro de comando de la catapulta

Los estados, comandos y configuraciones, por otra parte, se rigen a partir de las siguientes tablas:

Nombre	Descripción	Valor
`washer_status`	Apagada	0
`washer_status`	Encendida, puerta abierta	1
`washer_status`	Encendida, puerta cerrada	2
`washer_status`	Lavando	4
`washer_command`	Encender	1
`washer_command`	Apagar	255
`washer_command`	Abrir puerta	16
`washer_command`	Cerrar puerta	32
`washer_command`	Expulsar ropa	64
`washer_command`	Lavar	128

Nombre	Descripción	Valor
`dryer_status`	Apagada	0
`dryer_status`	Encendida, puerta abierta	1
`dryer_status`	Encendida, puerta cerrada	2
`dryer_status`	Secando	4
`dryer_command`	Encender	1
`dryer_command`	Apagar	255
`dryer_command`	Abrir puerta	16
`dryer_command`	Cerrar puerta	32
`dryer_command`	Expulsar ropa	64
`dryer_command`	Secar	128

Nombre	Descripción	Valor
`catapult_status`	Preparada	11
`catapult_status`	Disparada	13
`catapult_command`	Lanzar	11
`catapult_command`	Reposicionar	13
`washer_water_level`	Alto	1
`washer_water_level`	Medio	2
`washer_water_level`	Bajo	3
`dryer_type`	Ropa normal	1
`dryer_type`	Sábanas	2
`dryer_type`	Toallas	3

Para llevar a cabo la automatización, desarrolla el siguiente programa que asume que la ropa ya se encuentra dentro de la lavadora y que el lanzamiento de la catapulta es instantáneo:

```
washing_automation:
# Carga 0 + 0xFEE1DEAD en t0
addi t0, zero, 0xFEE1DEAD
addi s0, zero, 0
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 3
addi s4, zero, 4
addi s5, zero, 16
addi s6, zero, 32
addi s7, zero, 64
addi s8, zero, 128
addi s9, zero, 255
addi s10, zero, 11
addi s11, zero, 13

step_one:
    lw t1, 0(t0) # Carga Mem[t0 + 0] en t1
    # Salta a continue_step_one si t1 >= s1
    bge t1, s1, continue_step_one
    sw s1, 4(t0) # Carga s1 en Mem[t0 + 4]
    continue_step_one:
      # Salta a step_two si t1 == s2
      beq t1, s2, step_two
      sw s6, 4(t0)

step_two:
    lw t1, 24(t0)
    beq t1, s10, step_three
    sw s11, 28(t0)

# Continúa en step_three -->
```

```
step_three:
    lw t1, 12(t0)
    bge t1, s1, continue_step_three
    sw s1, 16(t0)
    continue_step_three:
      beq t1, s1, step_four
      sw s5, 16(t0)

step_four:
    sw s1, 8(t0)
    sw s8, 4(t0)
    while_step_four:
        lw t1, 0(t0)
        beq t1, s4, while_step_four
    sw s5, 4(t0)
    sw s7, 4(t0)
    sw s9, 4(t0)

step_five:
    sw s10, 28(t0)
    sw s11, 28(t0)

step_six:
    sw s6, 16(t0)
    sw s3, 20(t0)
    sw s8, 16(t0)
    while_step_six:
        lw t1, 12(t0)
        beq t1, s4, while_step_six
    sw s5, 16(t0)
    sw s7, 16(t0)
    sw s9, 16(t0)
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados, comandos y configuraciones de los dispositivos; no una explicación textual de cada instrucción. Sea preciso/a respecto a las condiciones evaluadas en los saltos e indique, cuando corresponda, el momento en el que se continúa al siguiente `step`.

Solución:

- `step_one` **(1 pto.)**: Se revisa el estado de la lavadora. Si está apagada, se prende. Luego, si está prendida con la puerta abierta, se cierra y salta a `step_two`. En otro caso, continúa a `step_two` directamente.
- `step_two` **(0.5 ptos.)**: Se revisa el estado de la catapulta. Si está preparada, se salta a `step_three`. En otro caso, se reposiciona y continúa a `step_three`.
- `step_three` **(1 pto.)**: Se revisa el estado de la secadora. Si está apagada, se prende. Luego, si está prendida con la puerta cerrada, se abre y salta a `step_four`. En otro caso, continúa a `step_four` directamente.
- `step_four` **(1.5 ptos.)**: Se configura un nivel alto de agua en la lavadora e inicia el lavado. Una vez que termina se abre la puerta, se expulsa la ropa (para que quede en la catapulta) y se apaga.
- `step_five` **(0.5 ptos.)**: Se lanza el contenido de la catapulta y se reposiciona.
- `step_six` **(1.5 ptos.)**: Se cierra la puerta de la secadora, se configura para toallas e inicia el secado. Una vez que termina se abre la puerta, se expulsa la ropa (para que quede en el canasto) y se apaga.

Para los pasos 1, 3, 4 y 6, se descuenta un 25 % del puntaje si existe como máximo un error o detalle faltante; un 50 % si existen como máximo dos errores; y un 100 % en otro caso. Para los pasos 2 y 5, se descuenta un 50 % del puntaje si existe como máximo un error o detalle faltante; y un 100 % en otro caso.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2025)

Interrogación 2

Pauta de evaluación

Pregunta 1: Representación de Números Reales (6 ptos.)

(a) (2 ptos.) Compare la representación de punto fijo con la representación de punto flotante en términos de rango y precisión. ¿Cuál es más conveniente en cada caso y por qué?

Solución: La representación de punto fijo es más conveniente si lo que se busca es **precisión**, dado que sus valores se distribuyen de manera uniforme en su rango gracias a la porción de bits de parte fraccional. No obstante, tiene un rango muy acotado que depende de los bits restantes para representar la parte entera del número. La representación de punto flotante, en cambio, es más conveniente si lo que se busca es un **rango mayor**, ya que el uso de bits para representar el exponente permite "flotar" el punto y representar valores mucho más grandes o pequeños. Hay que considerar, de todas formas, que mientras más grande o pequeño sea el valor a representar, su precisión disminuye significativamente al depender de los bits restantes para el significante. Se otorga **1 pto.** por describir correctamente en qué caso se prefiere una representación de punto fijo, y **1 pto.** por describir correctamente en qué caso se prefiere una representación de punto flotante.

(b) (1 pto.) Suponga que imprime en la consola de su computador un valor que se encuentra almacenado en formato float del estándar IEEE754. Al imprimir, en la consola ve que aparece el valor "`NaN`". ¿Es posible, a partir de este resultado, obtener la secuencia de 32 bits exacta que representa este valor? Justifique.

Solución: No es posible. La representación del valor `NaN` del formato float tiene las siguientes características:

- Su exponente es igual a 11111111.
- **Al menos uno de sus bits de significante es distinto de cero.**

El último punto es relevante. Dado que cualquiera de los bits del significante es distinto de cero, al leer en consola el valor "`NaN`" no tenemos cómo saber cuál de los 23 bits es igual a 1. Se otorgan **0.5 ptos.** por responder correctamente, y **0.5 ptos.** por justificación correcta. Si solo responde que no es posible sin ninguna justificación, no se otorga puntaje.

(c) (3 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

- (1.5 ptos.) $A + B$. $A =$ 0x40400000, $B =$ 0x80000000.

Solución:

- $A = 0$ 10000000 10000000000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 10000000b $\rightarrow$ 128 - 127 = 1.
  - **Significante:** 10000000000000000000000b $\rightarrow$ 1,1b.
  - **Valor:** $(1,1 \times 10^{1})$b = 11b = 3.
- $B = 1$ 00000000 00000000000000000000000b.
  - **Signo:** 1b $\rightarrow$ negativo.
  - **Exponente:** 00000000b $\rightarrow$ **reservado para el cero**.
  - **Significante:** 00000000000000000000000b $\rightarrow$ 1,0b.
  - **Valor:** -0 (valor reservado).

De lo anterior, se deduce que la operación a realizar es $3 + (-0)$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. No obstante, **no es necesario realizarla para deducir que el resultado es igual a 3** y, por ende, no hay pérdida de precisión al almacenar el valor de vuelta como float.

Se otorgan **0.375 ptos.** por la interpretación correcta de A; **0.375 ptos.** por la interpretación correcta de B; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Además, en este caso se acepta no realizar la operación si se deduce que la suma es por 0, manteniendo el valor final de A.

- (1.5 ptos.) $A \times B$. A = 0x40000000, B = 0xC0A00000.

Solución:

- A = 0 10000000 00000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000000b → 128 - 127 = 1.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $(1,0 \times 10^1)$b = 10b = 2.
- B = 1 10000001 01000000000000000000000b.
  - **Signo:** 1b → negativo.
  - **Exponente:** 10000001b → 129 - 127 = 2.
  - **Significante:** 01000000000000000000000b → 1,01b.
  - **Valor:** $-(1,01 \times 10^2)$b = $-$101b = -5.

De lo anterior, se deduce que la operación a realizar es 2×-5. En este caso, no se evaluará la forma en la que se realice la multiplicación; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se suman los exponentes sin desfase y se llega al valor $1 + 2 = 3 = 11$b. Luego, se multiplican los significantes: $1,0 \times -1,01 = -1,01$. Finalmente, se tiene como resultado $-1,01 \times 10^{11}$b = $-$1010b = $-$10, que en formato float se almacena como 11000001001000000000000000000000b = 0xC1200000. **No existe pérdida de precisión**.

Se otorgan **0.375 ptos.** por la interpretación correcta de A; **0.375 ptos.** por la interpretación correcta de B; **0.375 ptos.** por señalar el resultado correcto de la operación; y **0.375 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

Pregunta 2: Computador básico con subrutinas (6 ptos.)

En los ítems **(a)** y **(b)**, se le describirán instrucciones que deberá implementar en el computador básico con subrutinas a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura/incremento/decremento, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida ("-" si no afecta). Debe realizar todas las modificaciones de *hardware* necesarias en un solo diagrama. Además, puede implementar instrucciones sin modificar el *hardware* si es que la arquitectura se lo permite, indicando la forma de hacerlo a partir de las señales de control. **Cada instrucción debe ejecutarse en un ciclo**.

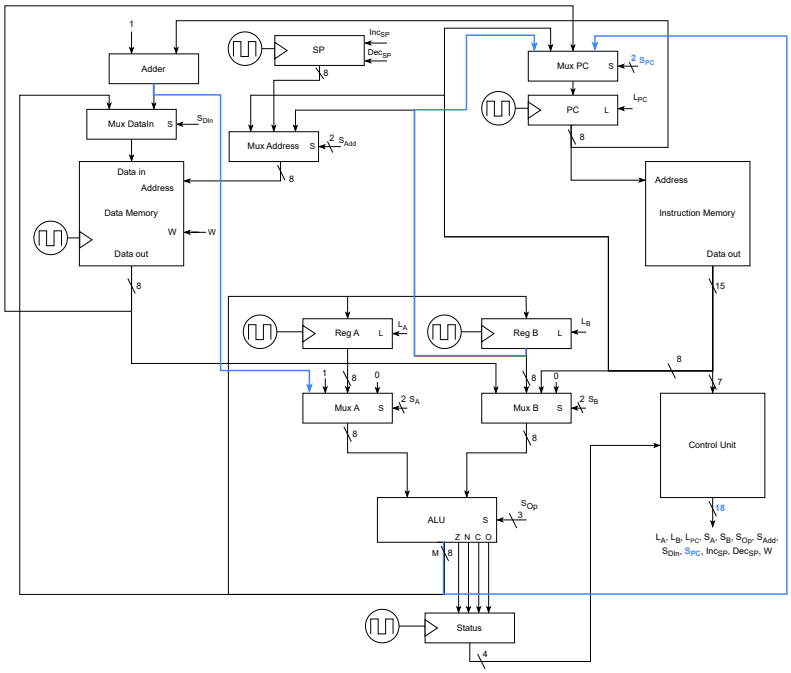
(a) (0.75 ptos.) `CALL A,B`. Almacena en el registro `A` la dirección de retorno `PC+1` y salta a la dirección de memoria almacenada en el registro `B`.

(b) (0.75 ptos.) `CALL B,Dir`. Almacena en el registro `B` la dirección de retorno `PC+1` y salta a la dirección de memoria equivalente al literal `Dir`.

(c) (0.75 ptos.) `RET A`. Salta a la dirección de retorno almacenada en el registro `A`.

(d) (0.75 ptos.) `RET B`. Salta a la dirección de retorno almacenada en el registro `B`.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Se conecta la salida del *Adder* que computa `PC+1` con la entrada restante del componente `Mux A`, de forma que se pueda tener este valor como salida de la ALU para su almacenamiento en los registros `A` o `B`.
- Se conecta la salida de la ALU con una de las entradas del componente `Mux PC`, de forma que se pueda seleccionar el valor del registro `A` o del registro `B` como direcciones de memoria a saltar.
- Se conecta la salida del registro `B` con una de las entradas del componente `Mux PC`, de forma que se pueda seleccionar el valor de este como dirección de memoria a saltar mientras que al mismo tiempo se compute `PC+1` desde la ALU.
- Finalmente, incrementa a 2 bits la señal S_{PC} para poder soportar las dos nuevas entradas del multiplexor `Mux PC`.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	Inc_{SP}	Dec_{SP}	W	S_A	S_B	S_{OP}	S_{Add}	S_{DIn}	S_{PC}
CALL A,B	1	0	1	0	0	0	PC+1	ZERO	ADD	-	-	B
CALL B,Dir	0	1	1	0	0	0	PC+1	ZERO	ADD	-	-	LIT
RET A	0	0	1	0	0	0	A	ZERO	ADD	-	-	ALU
RET B	0	0	1	0	0	0	ZERO	B	ADD	-	-	ALU

Por cada instrucción, se otorgan:

- **0.375 ptos.** por *hardware* correctamente modificado; **0.25 ptos.** si existe **un solo error** de implementación; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso.
- **0.375 ptos.** por entregar una combinación de señales correcta; **0.25 ptos.** si existe **un solo error**; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso. Si una instrucción se define en más de un ciclo, tampoco se otorga puntaje en este criterio. Si la instrucción se implementa correctamente sin modificación de *hardware*, se otorga el puntaje completo.

(e) (3 ptos.) A continuación, se adjuntan dos fragmentos de código **(1)** y **(2)** escritos en el Assembly del computador básico, con la diferencia de que **(2)** incorpora las instrucciones implementadas en los ítems anteriores. Para cada fragmento, indique los valores de los registros `A` y `B` al finalizar la ejecución del código. Si no incluye desarrollo o no justifica cómo llegó a los valores obtenidos, **no se otorgará puntaje**.

(1)

```
DATA:
  var1 12
  var2 4
CODE:
  MOV A,5     ; Dir. memoria 0x00
  MOV B,3     ; Dir. memoria 0x01
  RET         ; Dir. memoria 0x02-0x03
  case_1:
    ADD A,B   ; Dir. memoria 0x04
    PUSH A    ; Dir. memoria 0x05
    MOV A,B   ; Dir. memoria 0x06
    SHL A,A   ; Dir. memoria 0x07
    MOV B,A   ; Dir. memoria 0x08
    POP A     ; Dir. memoria 0x09-0x0A
    JMP end   ; Dir. memoria 0x0B
  case_2:
    SUB A,B   ; Dir. memoria 0x0C
    PUSH A    ; Dir. memoria 0x0D
    MOV A,B   ; Dir. memoria 0x0E
    SHR A,A   ; Dir. memoria 0x0F
    MOV B,A   ; Dir. memoria 0x10
    POP A     ; Dir. memoria 0x11-0x12
  end:
```

(2)

```
DATA:
  n    2
  res  0
CODE:
  MOV A,(n)       ; Dir. memoria 0x00
  CALL B,fun      ; Dir. memoria 0x01
  JMP end         ; Dir. memoria 0x02
  fun:
    CMP A,0       ; Dir. memoria 0x03
    JLE fun_end   ; Dir. memoria 0x04
    PUSH B        ; Dir. memoria 0x05
    MOV B,A       ; Dir. memoria 0x06
    ADD A,(res)   ; Dir. memoria 0x07
    MOV (res),A   ; Dir. memoria 0x08
    MOV A,B       ; Dir. memoria 0x09
    SUB A,1       ; Dir. memoria 0x0A
    CALL B,fun    ; Dir. memoria 0x0B
    POP B         ; Dir. memoria 0x0C-0x0D
    fun_end:
      RET B       ; Dir. memoria 0x0E

  end:
    MOV A,(res)   ; Dir. memoria 0x0F
```

Solución: El fragmento de código **(1)** accede, por *overflow* del registro `SP`, a la dirección de memoria 0 (equivalente a `var1`) para cargar su valor en el registro `PC`, por lo que la ejecución de `RET` genera el salto al *label* `case_2`. El fragmento de código **(2)**, por otra parte, computa de forma recursiva la suma de la variable `n` con todos sus antecesores hasta llegar a cero, almacenando el resultado final en `res`. A continuación, se muestra el resultado de la ejecución para ambos fragmentos:

(1)

```
DATA:
  var1 12     ; MEM[0] = 12
  var2 4      ; MEM[1] = 4
CODE:
  MOV A,5     ; A = 5
  MOV B,3     ; B = 3
  RET         ; SP++ => 255 + 1 = 0, PC = MEM[0] = 12
  case_1:
    ADD A,B   ; NO SE EJECUTA
    PUSH A    ; NO SE EJECUTA
    MOV A,B   ; NO SE EJECUTA
    SHL A,A   ; NO SE EJECUTA
    MOV B,A   ; NO SE EJECUTA
    POP A     ; NO SE EJECUTA
    JMP end   ; NO SE EJECUTA
  case_2:
    SUB A,B   ; A = 2
    PUSH A    ; MEM[SP] = A => MEM[0] = 2, SP-- => 0 - 1 = 255
    MOV A,B   ; A = 3
    SHR A,A   ; A = 1
    MOV B,A   ; B = 1
    POP A     ; SP++ => 255 + 1 = 0, A = MEM[0] = 2
  end: ; A = 2, B = 1
```

(2)

```
DATA:
  n    2          ; MEM[0] = 2
  res  0          ; MEM[1] = 0
CODE:
  MOV A,(n)       ; A = n = 2
  CALL B,fun      ; B = 2, fun(n)
  JMP end
  fun:
    CMP A,0
    JLE fun_end   ; Termina cuando A = 0
    PUSH B        ; Respalda B en stack (retorno)
    MOV B,A       ; B = A = n
    ADD A,(res)   ; A += res
    MOV (res),A   ; res = res + n
    MOV A,B       ; A = n
    SUB A,1       ; n--
    CALL B,fun    ; fun(n-1)
    POP B         ; Recupera B (retorno)
    fun_end:
      RET B       ; Retorna de la forma esperada
  end:
    ; A = res = fun(2) = 2+fun(1) = 2+1+fun(0) = 2+1+0 = 3
    MOV A,(res) ; B = Primer retorno computado = 2
```

Para cada fragmento de código, se otorgan **0.75 ptos.** por registro correctamente computado siempre que su cómputo esté respaldado por desarrollo o justificación. Si no se llega a ningún resultado final correcto por **error de arrastre**, se otorgan **0.5 ptos.** en el fragmento.

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

(a) (1 pto.) Señale una ventaja y una desventaja de mapear los dispositivos I/O de un computador en memoria.

Solución: Una ventaja es que no implica modificaciones en la ISA, lo que permite comunicarse directamente con los dispositivos a través de instrucciones de lectura y escritura de memoria. Una desventaja, en cambio, es el uso significativo de direcciones de memoria a reservar, que en memorias más pequeñas puede implicar problemas de ejecución en los programas (*memory barrier*). Se otorgan **0.5 ptos.** por describir correctamente una ventaja y **0.5 ptos.** por describir correctamente una desventaja.

(b) (1 pto.) Explique el uso e importancia de las ISRs ejecutadas por la CPU en el contexto de comunicación vía interrupciones.

Solución: Las ISRs o *Interrupt Service Routines* son subrutinas asociadas al manejo que requieren los dispositivos por parte de la CPU una vez que son atendidas por la CPU. Son importantes ya que, a través de su ejecución, se lleva a cabo la comunicación entre la CPU y los dispositivos. Se otorgan **0.5 ptos.** por describir correctamente en qué consiste una ISR y **0.5 ptos.** por destacar su importancia.

(c) (4 ptos.) Suponga que decide automatizar la disponibilidad de un baño mediante sensores y un emisor Bluetooth conectados a un computador con arquitectura RISC-V. Para la comunicación con los sensores y el emisor, sus registros de 32 bits están mapeados en memoria desde la dirección `0x600DD0D0`:

Offset	**Nombre**	**Descripción**
0x00	`door_sensor_status`	Registro estado sensor de puerta
0x04	`light_sensor_status`	Registro estado sensor de luz
0x08	`notifier_command`	Registro comando emisor
0x0C	`notifier_status`	Registro estado emisor

Nombre	**Descripción**	**Valor**
`door_sensor_status`	Puerta cerrada	0
`door_sensor_status`	Puerta abierta	1
`light_sensor_status`	Luz apagada	0
`light_sensor_status`	Luz encendida	1
`notifier_status`	Inactivo	0
`notifier_status`	Activo	1
`notifier_status`	Enviando	2
`notifier_command`	Encender emisor	1
`notifier_command`	Notificar `OPEN`	2
`notifier_command`	Notificar `CLOSED`	3
`notifier_command`	Apagar emisor	4

El sistema debe enviar una notificación Bluetooth según la disponibilidad del baño: `OPEN` si es que está disponible, y `CLOSED` si es que está ocupado según los estados de los sensores de puerta (`door_sensor_status`) y luz (`light_sensor_status`). Para ello, se desarrolla el siguiente programa:

```
notifier_automation:
    # Carga zero + 0x600DD0D0 en t0
    addi t0, zero, 0x600DD0D0
    addi s0, zero, 0
    addi s1, zero, 1
    addi s2, zero, 2
    addi s3, zero, 3
    addi s4, zero, 4

step_one:
    lw t3, 12(t0) # Carga Mem[t0 + 12] en t3
    # Salta a step_two si t3 != s0
    bne t3, s0, step_two
    sw s1, 8(t0) # Carga s1 en Mem[t0 + 8]

step_two:
    lw t1, 0(t0)
    lw t2, 4(t0)
    bne t1, s1, step_three
    bne t2, s0, step_three
    # Salto incondicional a step_four
    jal zero, step_four
```

```
# => Continuación

step_three:
    sw s3, 8(t0)

    while_step_three:
        lw t3, 12(t0)
        # Salta a while_step_three si t3 == s2
        beq t3, s2, while_step_three

    sw s4, 8(t0)
    jal zero, step_one

step_four:
    sw s2, 8(t0)

    while_step_four:
        lw t3, 12(t0)
        beq t3, s2, while_step_four

    sw s4, 8(t0)
    jal zero, step_one
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(1 pto.)**: Se revisa el estado del emisor. Si el emisor está inactivo, se enciende y continúa a `step_two`. En otro caso, salta directamente a `step_two`.
- `step_two` **(1 pto.)**: Se revisa el estado del sensor de puerta y el sensor de luz del baño. Si la puerta está cerrada o la luz está encendida, salta a `step_three`. En otro caso, salta directamente a `step_four`.
- `step_three` **(1 pto.)**: A través del emisor, se envía la notificación `CLOSED`. Luego, se espera a que termine el envío y se apaga el emisor. Finalmente, salta de vuelta al `step_one`.
- `step_four` **(1 pto.)**: A través del emisor, se envía la notificación `OPEN`. Luego, se espera a que termine el envío y se apaga el emisor. Finalmente, salta de vuelta al `step_one`.

Para cada paso, se descuenta un 25 % del puntaje si existe como máximo un error o detalle faltante; un 50 % si existen como máximo dos errores; y un 100 % en otro caso.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Interrogación 2 - Parte 2 - Sección 1

Pauta de evaluación

Pregunta 2: Representación de Números Reales (6 ptos.)

(a) (2 ptos.) Indique cuántos bits de precisión posee la representación de un número float del estándar IEEE754 y explique por qué.

> **Solución:** La representación de floats del estándar IEEE754 cuenta con **24 bits de precisión**. Esto se debe a la inclusión del bit implícito, el que otorga un bit de precisión adicional a los 23 bits del significante. Se otorga **1 pto.** por señalar la cantidad correcta de bits de precisión y **1 pto.** por justificación.

(b) (4 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

- (2 ptos.) $A + B$. $A =$ 0xBF800000, $B =$ 0x7F800008.

> **Solución:**
>
> - $A =$ 1 01111111 00000000000000000000000b.
> - **Signo:** 1b → negativo.
> - **Exponente:** 01111111b → 127 - 127 = 0.
> - **Significante:** 00000000000000000000000b → 1,0b.
> - **Valor:** $-(1,0 \times 10^0)$b $= -1$b $= -1$.
> - $B =$ 0 11111111 00000000000000000001000b.
> - **Signo:** 0b → positivo.
> - **Exponente:** 11111111b → **reservado para infinito y `NaN`**.

- **Significante:** 00000000000000000001000b → distinto de $1,0$.
- **Valor:** `NaN` (valor reservado).

De lo anterior, se deduce que la operación a realizar es $-1 + \texttt{NaN}$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. No obstante, **no es necesario realizarla para deducir que el resultado es igual a `NaN`** y, por ende, no hay pérdida de precisión al almacenar el valor de vuelta como float. En este caso, en estricto rigor la representación en formato float del resultado podría o no ser igual a B ya que depende de la implementación, por lo que se considera correcto si deducen que el resultado es `NaN` independiente de la representación que utilicen.

Se otorgan **0.5 ptos.** por la interpretación correcta de A; **0.5 ptos.** por la interpretación correcta de B; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no. Además, en este caso se acepta no realizar la operación si se deduce que la suma es por `NaN`, manteniendo el valor final de B.

- (2 ptos.) $A \times B$. $A =$ 0xC0000000, $B =$ 0x42000000.

Solución:

- $A =$ 1 10000000 00000000000000000000000b.
  - **Signo:** 1b → negativo.
  - **Exponente:** 10000000b → 128 - 127 = 1.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $-(1,0 \times 10^1)$b $= -10$b $= -2$.
- $B =$ 0 10000100 00000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 10000100b → 132 - 127 = 5.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $(1,0 \times 10^5)$b $= 100000$b $= 32$.

De lo anterior, se deduce que la operación a realizar es -2×32. En este caso, no se evaluará la forma en la que se realice la multiplicación; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se suman los exponentes sin desfase y se llega al valor $1+5 = 6 = 110$b. Luego, se multiplican los significantes: $-1,0 \times 1,0 = -1,0$. Finalmente, se tiene como resultado $-1,0 \times 10^{110}$b $= -1000000$b $= -64$, que en formato float se almacena como 11000010100000000000000000000000b = 0xC2800000. **No existe pérdida de precisión**.

Se otorgan **0.5 ptos.** por la interpretación correcta de A; **0.5 ptos.** por la interpretación correcta de B; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

(a) (1.5 ptos.) Explique el significado y uso del identificador que envía el controlador de interrupciones luego de recibir la señal INTA por parte de la CPU.

Solución: Al recibir la señal INTA, la CPU espera obtener el identificador asociado al **dispositivo cuya interrupción será atendida**. Este será enviado por el controlador de interrupciones. Luego, la CPU hace uso de este valor como índice del vector de interrupciones para obtener el puntero de la ISR del dispositivo atendido, permitiendo así la ejecución de su subrutina. Se otorgan **0.75 ptos.** por indicar la asociación entre el identificador y los dispositivos I/O; y **0.75 ptos.** por indicar su uso en la ejecución de la ISR deseada.

(b) (1.5 ptos.) Indique qué diferencia existe entre una interrupción gatillada por un dispositivo I/O y una gatillada por *software*.

Solución: La **única** diferencia es la señalada en el enunciado: una se gatilla desde un dispositivo I/O, mientras que la otra se gatilla por *software* (ya sea por una excepción o por un *trap*). En ambos casos, la interrupción se lleva a cabo a través de la ejecución de una ISR cuyo puntero se almacena en el vector de interrupciones, y ambas cuentan con un identificador que envía el controlador de interrupciones al recibir la señal INTA por parte de la CPU. Se otorgan **0.75 ptos.** por señalar la fuente de la interrupción como diferencia, y **0.75 ptos.** por explicar por qué no se diferencian en nada más.

(c) (3 ptos.) Suponga que decide, con el propósito de ahorrar tiempo, automatizar la preparación de café filtrado a partir de un molinillo y hervidor eléctricos conectados a un computador con ISA RISC-V. A continuación, una representación gráfica de su implementación:



Para la comunicación con los electrodomésticos, sus registros de 32 bits son mapeados en memoria desde la dirección 0x600DCAFE:

Offset	Nombre	Descripción
0x00	`coffee_grinder_status`	Registro estado molinillo
0x04	`coffee_grinder_command`	Registro comando molinillo
0x08	`coffee_grinder_grind_size`	Config. grosor molienda
0x0C	`coffee_grinder_weight`	Config. gramos a moler
0x10	`kettle_status`	Registro estado hervidor
0x14	`kettle_command`	Registro comando hervidor
0x18	`kettle_temperature`	Config. temperatura hervidor (ºC)

Nombre	Descripción	Valor
`coffee_grinder_status`	Apagado	0
`coffee_grinder_status`	Encendido	1
`coffee_grinder_status`	Moliendo	2
`kettle_status`	Apagado	0
`kettle_status`	Encendido	1
`kettle_status`	Hirviendo	2
`coffee_grinder_command`	Encender	1
`coffee_grinder_command`	Apagar	255
`coffee_grinder_command`	Moler	128
`kettle_command`	Encender	1
`kettle_command`	Apagar	255
`kettle_command`	Hervir	128
`coffee_grinder_grind_size`	Fino	1
`coffee_grinder_grind_size`	Medio	2
`coffee_grinder_grind_size`	Grueso	3

Pensando en su preparación favorita, desarrolla el siguiente programa:

```
coffee_system:
# Carga 0 + 0x600DCAFE en t0
addi t0, zero, 0x600DCAFE
addi t2, zero, 0
addi t3, zero, 0
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 128
addi s4, zero, 255
addi s5, zero, 30
addi s6, zero, 92

step_one:
    lw t1, 0(t0) # Carga Mem[t0 + 0] en t1
    # Salta a continue_step_one si t1 != s2
    bne t1, s2, continue_step_one
    jal ra, step_five # Llamado a step_five
    beq zero, zero, step_three
    continue_step_one:
        beq t1, s1, step_two
        sw s1, 4(t0) # Carga s1 en Mem[t0 + 4]
step_two:
    sw s2, 8(t0)
    sw s5, 12(t0)
    sw s3, 4(t0)
    jal ra, step_five
# Continúa en step_three -->
```

```
step_three:
    lw t1, 16(t0) # Carga Mem[t0 + 16] en t1
    # Salta a continue_step_three si t1 != s2
    bne t1, s2, continue_step_three
    jal ra, step_six
    beq zero, zero, end # Salto incondicional
    continue_step_three:
        beq t1, s1, step_four
        sw s1, 20(t0)
step_four:
    sw s6, 24(t0)
    sw s3, 20(t0)
    jal ra, step_six
    beq zero, zero, end
step_five:
    while_step_five:
        lw t1, 0(t0)
        beq t1, s2, while_step_five
    sw s4, 4(t0)
    jalr zero, 0(ra) # Retorno
step_six:
    while_step_six:
        lw t1, 16(t0)
        beq t1, s2, while_step_six
    sw s4, 20(t0)
    jalr zero, 0(ra)
end: # Término del programa.
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(0.5 ptos.)**: Se revisa el estado del molinillo. Si está moliendo, se ejecuta `step_five` y se salta a `step_three`. Si está encendida, se salta a `step_two`. En otro caso, se enciende antes de continuar con `step_two`.
- `step_two` **(0.5 ptos.)**: Se configura el molinillo con una molienda de grosor medio y un total de 30 gramos. Luego, empieza a moler y se ejecuta `step_five` antes de seguir a `step_three`.
- `step_three` **(0.5 ptos.)**: Se revisa el estado del hervidor. Si está hirviendo, se ejecuta `step_six` y se termina el programa. Si está encendida, se salta a `step_four`. En otro caso, se enciende antes de continuar con `step_four`.
- `step_four` **(0.5 ptos.)**: Se configura el hervidor con una temperatura de 92ºC. Luego, empieza a hervir y se ejecuta `step_six` antes de terminar el programa.
- `step_five` **(0.5 ptos.)**: Se revisa el estado del molinillo hasta que deje de moler.
- `step_six` **(0.5 ptos.)**: Se revisa el estado del hervidor hasta que deje de hervir.

Para cada `step`, se otorga la mitad del puntaje si existe como máximo un error o detalle faltante. Si existen dos o más errores o detalles faltantes, no se otorga puntaje.

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (I/2026)

Interrogación 2 - Parte 2 - Sección 2

Pauta de evaluación

Pregunta 2: Representación de Números Reales (6 ptos.)

(a) (2 ptos.) Indique cuántos exponentes distintos se pueden representar en un número float del estándar IEEE754 y explique por qué.

Solución: Como los exponentes 00000000b y 11111111b se reservan para las representaciones de 0 y $\pm\infty$/`NaN` respectivamente, el rango de exponentes de los floats bajo el estándar IEEE754 corresponde a $[-126, 127]$, es decir, se tienen 254 exponentes distintos (256 combinaciones distintas - 2 reservadas). Se otorga **1 pto.** por indicar correctamente la cantidad de exponentes distintos, y **1 pto.** por justificación.

(b) (4 ptos.) En los siguientes incisos, se le entregará un par de números reales A, B en formato float del estándar IEEE754 con notación hexadecimal. En cada caso, debe interpretar sus valores y realizar las operaciones de suma o multiplicación solicitadas. Luego, debe indicar el valor del resultado de la operación y señalar si se pierde precisión al almacenarlo de vuelta como float del estándar IEEE754, indicando además cómo queda almacenado en este formato. Los valores deben presentarse en base binaria con notación científica normalizada.

- (2 ptos.) $A + B$. A = 0x58800000, B = 0x3F800000.

Solución:

- A = 0 10110001 0000000 00000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.
  - **Exponente:** 10110001b $\rightarrow$ 177 - 127 = 50.
  - **Significante:** 00000000000000000000000b $\rightarrow$ 1,0b.
  - **Valor:** $(1,0 \times 10^{50})$b = 10000..,000b = 2^{50}.
- B = 0 01111111 00000000000000000000000b.
  - **Signo:** 0b $\rightarrow$ positivo.

- **Exponente:** 01111111b → 127 - 127 = 0.
- **Significante:** 00000000000000000000000b → 1,0b.
- **Valor:** $(1,0 \times 10^0)$b = 1b = 1.

De lo anterior, se deduce que la operación a realizar es $2^{50} + 1$. En este caso, no se evaluará la forma en la que se realice la suma; solo que su resultado sea correcto. Aquí se opta por seguir el algoritmo visto en clases: se igualan los exponentes y B se expresa como $0,0000..,001 \times 10^{50}$. Luego, sumando los significantes, se tiene $A + B = (1,0000..,001 \times 10^{50})$b. Finalmente, si se almacena como `float`, se tiene el valor 01011000100000000000000000000000b = 0x58800000 = 2^{50}. **Se pierden los 27 bits menos significantes que aportaban la magnitud de 1 y, por ende, se pierde precisión**.

Se otorgan **0.5 ptos.** por la interpretación correcta de A; **0.5 ptos.** por la interpretación correcta de B; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no. Se acepta escribir el valor en potencias de 2.

- (2 ptos.) $A \times B$. A = 0xBF800000, B = 0x7F800000.

Solución:

- A = 1 01111111 00000000000000000000000b.
  - **Signo:** 1b → negativo.
  - **Exponente:** 01111111b → 127 - 127 = 0.
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** $-(1,0 \times 10^0)$b = −1b = −1.
- B = 01111111100000000000000000000000b.
  - **Signo:** 0b → positivo.
  - **Exponente:** 11111111b → **reservado para infinito y `NaN`**
  - **Significante:** 00000000000000000000000b → 1,0b.
  - **Valor:** ∞.

De lo anterior, se deduce que la operación a realizar es igual a $-1 \times \infty$. En este caso, no se evaluará la forma en la que se realice la multiplicación; solo que su resultado sea correcto. No obstante, **no es necesario realizarla para deducir que el resultado es igual a** $-\infty$, cuya representación en float es igual a 11111111100000000000000000000000b = 0xFF800000. Por lo tanto, no hay pérdida de precisión al almacenar el valor de vuelta como float.

Se otorgan **0.5 ptos.** por la interpretación correcta de A; **0.5 ptos.** por la interpretación correcta de B; **0.5 ptos.** por señalar el resultado correcto de la operación; y **0.5 ptos.** por indicar correctamente si se pierde precisión o no.

Pregunta 3: Comunicación con dispositivos I/O (6 ptos.)

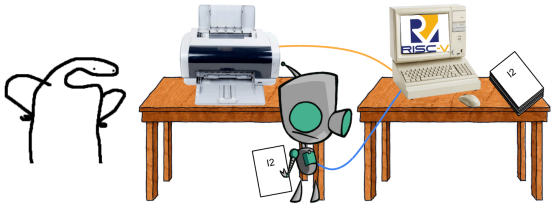
(a) (1.5 ptos.) ¿Qué pasaría, en el contexto de la comunicación con dispositivos I/O a través de interrupciones, si se eliminara la *flag* IF? Justifique su respuesta.

> **Solución:** La *Interruption Flag* le indica a la CPU si puede o no atender interrupciones. Si se elimina, la CPU no contaría con un mecanismo para determinar si debe detener su ejecución y atender una interrupción, impidiendo así la comunicación vía interrupciones. Otro posible escenario sería que se atiendan interrupciones aunque ya haya una en ejecución. Esto implicaría que no haya control en la ejecución anidada de ISRs y que se generen retrasos en su término (o incluso *stack overflow*). Se otorgan **0.75 ptos.** por señalar un posible escenario al eliminar la *flag*, y **0.75 ptos.** por justificación.

(b) (1.5 ptos.) ¿Cuál es la función del vector de interrupciones? ¿Cuál es su contenido?

> **Solución:** La función del vector de interrupciones es permitir la ejecución correcta de la ISR (*Interruption Service Routine*) a partir del identificador del dispositivo a ser atendido, lo que es gestionado por el controlador de interrupciones. Este vector contiene los **punteros** de las ISR, por lo que si se desea atender al dispositivo `i`, se ejecuta el llamado de la subrutina a partir de su dirección igual a `vector_interrupciones[i]`. Se otorgan **0.75 ptos.** por señalar correctamente la función del vector de interrupciones; y **0.75 ptos.** por indicar de forma correcta su contenido.

(c) (3 ptos.) Suponga que es el/la docente del curso y debe imprimir la I2. No obstante, la redacta a última hora y no alcanza a usar los equipos del DCC, por lo que decide imprimir en su hogar con un sistema innovador: una impresora y un robot conectados a un computador con ISA RISC-V. A continuación, una representación gráfica de su implementación:



Para llevar a cabo la comunicación, los registros de 32 bits de los dispositivos se mapean en memoria desde la dirección 0x00FF10AD:

Offset	Nombre	Descripción
0x00	`printer_status`	Registro de estado impresora
0x04	`printer_command`	Registro de comando impresora
0x08	`printer_copies`	Configuración de cantidad de copias impresora
0x0C	`printer_color`	Configuración de color impresora
0x10	`robot_status`	Registro de estado robot
0x14	`robot_command`	Registro de comando robot

Nombre	Descripción	Valor
`printer_status`	Apagado	0
`printer_status`	Encendido	1
`printer_status`	Imprimiendo	2
`robot_status`	Apagado	0
`robot_status`	Encendido	1
`robot_status`	Trabajando	2
`printer_command`	Encender	1
`printer_command`	Apagar	255
`printer_command`	Imprimir	128
`printer_color`	Blanco y negro	1
`printer_color`	Color	2
`robot_command`	Encender	1
`robot_command`	Rellenar papeles impresora	2
`robot_command`	Rellenar tinta impresora	3
`robot_command`	Apilar impresiones*	4
`robot_command`	Apagar	255

* Esta acción hace que el robot pase a estado “Trabajando”, el resto son instantáneas y no cambian su estado.

Para automatizar la impresión, desarrolla el siguiente programa:

```
printing_system:
# Carga 0 + 0x00FF10AD en t0
addi t0, zero, 0x00FF10AD
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 3
addi s4, zero, 4
addi s5, zero, 128
addi s6, zero, 255
addi s7, zero, 60

step_one:
    lw t1, 16(t0) # Carga Mem[t0 + 16] en t1
    # Salta a continue_step_one si t1 != zero
    bne t1, zero, continue_step_one
    sw s1, 20(t0) # Carga s1 en Mem[t0 + 20]
    continue_step_one:
        lw t1, 16(t0)
        # Salta a continue_step_one si t1 == s2
        beq t1, s2, continue_step_one

step_two:
    lw t1, 0(t0)
    bne t1, zero, continue_step_two
    sw s1, 4(t0) # ---> Continúa a la derecha
```

```
# --> Continúa aquí
    continue_step_two:
        lw t1, 0(t0)
        beq t1, s2, continue_step_two

step_three:
    sw s7, 8(t0)
    sw s3, 20(t0)
    sw s1, 12(t0)

step_four:
    sw s5, 4(t0)
    continue_step_four:
        lw t1, 0(t0)
        beq t1, s2, continue_step_four

step_five:
    sw s4, 20(t0)
    continue_step_five:
        lw t1, 16(t0)
        beq t1, s2, continue_step_five

step_six:
    sw s6, 4(t0)
    sw s6, 20(t0) # Término del programa.
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- `step_one` **(0.6 ptos.)**: Se revisa el estado del robot. Si está apagado, se enciende antes de seguir. Luego, si está trabajando, se revisa su estado hasta verificar que terminó de trabajar.
- `step_two` **(0.6 ptos.)**: Se revisa el estado de la impresora. Si está apagada, se enciende antes de seguir. Luego, si está imprimiendo, se revisa su estado hasta verificar que terminó de imprimir.
- `step_three` **(0.4 ptos.)**: Se configura un total de 60 copias de impresión; se configura la impresión en blanco y negro; y el robot rellena las tintas de la impresora.
- `step_four` **(0.5 ptos.)**: Se inicia la impresión y se revisa el estado de la impresora hasta que termine de imprimir.
- `step_five` **(0.5 ptos.)**: Se le instruye al robot que apile las impresiones y se espera a que termine de apilarlas.
- `step_six` **(0.4 ptos.)**: Se apaga tanto la impresora como el robot y termina el programa.

Para cada `step`, se otorga la mitad del puntaje si existe como máximo un error o detalle faltante. Si existen dos o más errores o detalles faltantes, no se otorga puntaje.